AI DROWSINESS DETECTION SYSTEM

Main-Project report submitted to
Kristu Jyoti College of Management and Technology,
Changanacherry

In partial fulfillment of the requirements for the award of the

BACHELOR OF COMPUTER APPLICATION

BY

ANAND S 230021080179

Under the guidance of
MS. ABY ROSE VARGHESE

DEPARTMENT OF COMPUTER APPLICATIONS

KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY, CHANGANACHERRY

APRIL, 2026

KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY, CHANGANACHERRY

DEPARTMENT OF COMPUTER APPLICATIONS

CERTIFICATE

Certify that the project report entitled **"AI DROWSINESS DETECTION SYSTEM"** is a bonafide report of the project done by **ANAND S (230021080179**) under our guidance and supervision is submitted in partial fulfillment of the Bachelor of Computer Applications, awarded by Mahatma Gandhi University, Kerala and that no part of this work has been submitted earlier for the award of any other degree.

Rev.Dr. Joshy George **Mr. Roji Thomas**

(Principal) **(HOD)**

Department Seal

Ms.Aby Rose Varghese **Ms.Tintu Varghese**

(Project Guide) **(Project Coordinator)**

Submitted for presentation and viva held on……

Examiner(s) :

1.

2.

GLACE BYTE
www.glacebyte.in

8139802775
glacebyte@gmail.com
Kottayam, 686001

CERTIFICATE OF PROJECT COMPLETION

Apr 13, 2026

This is proudly certify that:

ANAND S
Register No : 230021080179

has successfully completed main project titled

AI Drowsiness Detection System

as part of the academic requirements for the Bachelor of Computer Applications (BCA) degree of **Kristu Jyoti College of Management & Technology** , during the academic year 2025-2026.
This project was developed using Python, OpenCV, dlib, NumPy, SciPy, pyttsx3, and other supporting libraries under the technical guidance of **Glace Byte IT Solutions** , Kottayam.
The project demonstrates commendable effort, innovation, and technical understanding in applying theoretical concepts to a real-world software solution. The student has shown proficiency in computer vision, real-time monitoring, alert system integration, logging, and software design principles, contributing to a well-structured and functional system.
We appreciate the student's commitment, analytical approach, and dedication throughout the development and documentation phases of the project.

Project Guide

GLACE BYTE IT SOLUTIONS
2nd FLOOR
ASHOKA CHAMBERS
SHASTRI ROAD
Ph: 8139802775
KOTTAYAM-686001

NITHIN P S
GLACE BYTE IT SOLUTIONS

DECLARATION

I hereby declare that the project work entitled "**AI DROWSINESS DETECTION SYSTEM**" submitted to Mahatma Gandhi University in partial fulfillment of requirement for the award of degree of Bachelor of Computer Application from Kristu Jyoti College of Management and Technology, Changanacherry is a record of bonafide work done under the guidance of **Ms.ABY ROSE VARGHESE**, project guide, Kristu Jyoti of Management and Technology, Changanacherry. This project has not been submitted in partial or fulfillment of any other degree/diploma/fellowship or similar of this university or any other university.

Place : Changanacherry — Anand S

Date : — 230021080179

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to several people who have been instrumental in the successful completion of this project. First of all, I thank the college management for providing the necessary facilities and support throughout the project work.

I am greatly indebted to Rev. Dr. Joshy George, Principal, Kristu Jyoti College, for granting me permission to carry out this project and for his valuable encouragement.

I record my sincere thanks and gratitude to Mr. Roji Thomas, HOD, Department of Computer Applications, for his valuable advice and encouragement during the course of this project.

I am extremely grateful to Ms. Aby Rose Varghese, Project Guide, and Ms. Tintu Varghese, Project Coordinator, for their valuable guidance and suggestions, which helped me overcome the major challenges encountered during the completion of this project.

Anand S

230021080179

INDEX

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Introduction

Road accidents caused by driver fatigue and drowsiness have become a major global concern in recent years. Drowsy driving significantly reduces a driver's alertness, reaction time, and decision-making ability, often leading to severe accidents, injuries, and loss of life. According to various road safety studies, driver drowsiness is one of the leading causes of highway accidents, especially during long-distance travel and night driving.

Traditional vehicle safety systems mainly depend on hardware-based solutions such as sensors, alarms, or wearable devices. However, these systems increase cost, require additional hardware installation, and may not always provide timely alerts. With advancements in Artificial Intelligence and Machine Learning, it has become possible to detect driver drowsiness using computer vision techniques without relying on extra hardware.

This project presents an AI-Based Drowsiness Detection System developed using Python and Machine Learning, which operates purely through software-based methods. The system continuously monitors the driver's facial features through a real-time camera feed and detects signs of drowsiness such as prolonged eye closure and yawning. When drowsiness is detected, the system triggers an alert sound to warn the driver and help prevent potential accidents.

The proposed system is cost-effective, non-intrusive, and easy to deploy, making it suitable for real-world applications such as intelligent transportation systems and driver monitoring solutions.

1.2 Problem Statement

Driver drowsiness is difficult to detect manually and often goes unnoticed until it results in an accident. Existing methods either depend on self-awareness of the driver or require specialized hardware such as sensors or EEG devices, which are expensive and inconvenient for everyday use.

Manual monitoring is unreliable, and hardware-based systems may fail due to sensor errors, discomfort, or maintenance issues. There is a strong need for an automated, real-time, and software-driven system that can accurately detect driver fatigue using visual cues and provide timely alerts to reduce accident risks.

1.3 Scope and Relevance of the Project

The scope of this project includes the development of a real-time drowsiness detection system using computer vision and facial landmark analysis. The system uses a standard webcam to monitor the driver's face and analyzes eye and mouth movements to determine alertness levels.

The project focuses on:

- Eye blink rate and eye closure detection
- Yawn detection using mouth aspect ratio
- Real-time alert generation using an alarm sound
- Distraction Monitoring

The relevance of this project lies in improving road safety by providing an intelligent and automated driver monitoring solution. The system can be implemented in vehicles, driving schools, transport fleets, and research environments. Since the system does not require additional hardware, it is scalable and suitable for widespread adoption.

1.4 Objectives

The main objectives of this project are:

1. To develop a real-time AI-based drowsiness detection system using Python.

2. To detect driver fatigue by analyzing eye aspect ratio and mouth movements.

3. To implement yawn detection as an additional indicator of drowsiness.

4. To provide an alert mechanism that warns the driver when drowsiness is detected.

5. To build a software-only solution without using external hardware sensors.

6. To ensure real-time processing using computer vision techniques.

7. To design a cost-effective and non-intrusive safety system.

8. To demonstrate the practical application of AI and Machine Learning in accident prevention.

SYSTEM ANALYSIS

CHAPTER – 2

SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis is the process of studying an existing system, identifying its problems, and designing an improved system.

In this project, an **AI-Based Drowsiness Detection System** is developed using Python, Machine Learning, and Computer Vision techniques. The system continuously monitors the driver through a real-time camera feed and detects signs such as:

- Eye closure using Eye Aspect Ratio (EAR)
- Yawning using Mouth Aspect Ratio (MAR)
- Driver distraction (face not focused / looking away)

The system also maintains a session log to record events such as drowsiness, yawning, and distraction. Based on these events, a risk indicator is generated to assess the driver's alertness level.

When drowsiness or distraction is detected, the system triggers an audio alert to warn the driver and prevent accidents.

The system is software-based, cost-effective, and non-intrusive.

2.2 EXISTING SYSTEM

Existing methods for monitoring driver alertness include:

- Driver self-assessment
- Manual supervision
- Road safety signs
- Basic vehicle alarms
- Hardware-based fatigue detection

These methods lack intelligent real-time monitoring and do not analyze driver behavior using AI.

2.2.1 LIMITATIONS OF EXISTING SYSTEM

- No real-time continuous monitoring
- Dependence on driver awareness
- Expensive hardware requirements
- No behavioral analysis using AI

- No tracking of driver activity
- No risk evaluation system

2.3 PROPOSED SYSTEM

The proposed system is an AI-powered real-time driver monitoring system **that uses computer vision to detect drowsiness and distraction.**The system captures live video and analyzes facial landmarks:

- Eyes → for blink and closure detection
- Mouth → for yawning detection
- Face direction → for distraction detection

It calculates:

- Eye Aspect Ratio (EAR)
- Mouth Aspect Ratio (MAR)

Additional Functionalities

- Distraction Detection: Identifies if the driver is not focusing on the road
- Session Monitoring: Records events such as eye closure, yawning, and distraction
- Risk Indicator: Calculates a risk level based on detected events

If abnormal behavior is detected, the system generates an **audio** alert**.**

Key Features of the Proposed System

- Real-time face detection
- Eye closure detection
- Yawning detection
- Distraction detection
- Audio alert system
- Session logging
- Risk level calculation
- Continuous monitoring
- Software-based implementation

2.3.1 ADVANTAGES OF PROPOSED SYSTEM

- Automation
- High Accuracy
- Real-Time Monitoring
- Cost-Effective
- Non-Intrusive

- Enhanced Safety
- Behavior Tracking
- Risk Awareness

2.4 FEASIBILITY STUDY

A feasibility study is conducted to evaluate whether the proposed system can be successfully developed and implemented.

2.4.1 TECHNICAL FEASIBILITY

The system is technically feasible because:

- Python supports powerful computer vision and machine learning libraries
- OpenCV enables real-time video processing
- Dlib provides accurate facial landmark detection
- Required hardware (camera and computer) is easily available
- The system can run on standard operating systems

Hence, the system can be developed using existing and reliable technologies.

2.4.2 OPERATIONAL FEASIBILITY

The system is operationally feasible because

- It is easy to use and requires minimal training.
- The driver does not need to perform any special actions.
- Alerts are generated automatically.
- The system operates in real time without user intervention.

Therefore, the system can be easily adopted in real-world driving environments.

2.4.3 ECONOMIC FEASIBILITY

The system is economically feasible because:

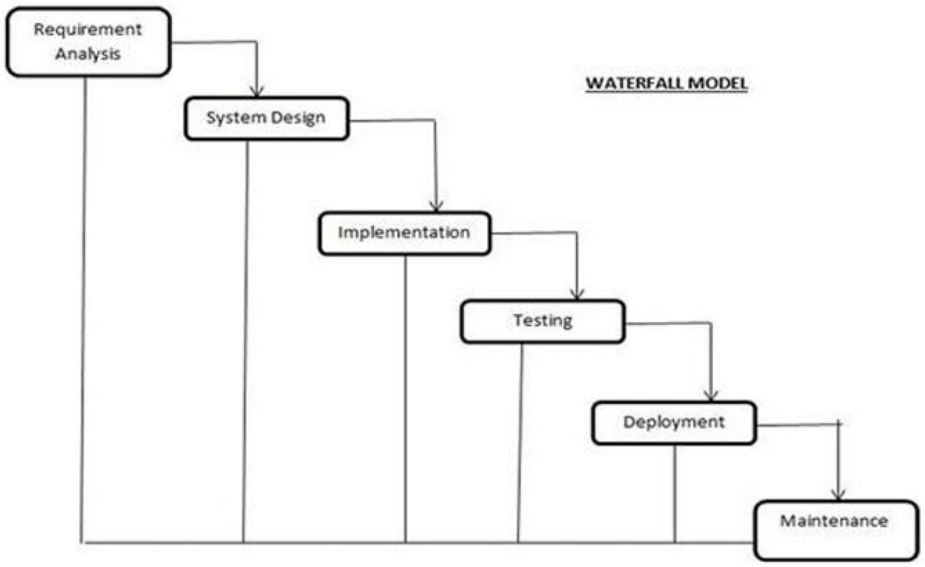
- It uses open-source tools like Python and OpenCV
- No expensive sensors or hardware are required
- Development and maintenance costs are very low
- It reduces accident-related financial losses

Thus, the system is affordable and cost-effective.

2.5 SOFTWARE ENGINEERING PARADIGM APPLIED

The Waterfall Model of the Software Development Life Cycle (SDLC) is applied in this project. The stages include:

- Requirement Analysis: Understanding system requirements
- System Design: Designing system architecture
- Implementation: Coding using Python and AI libraries
- Testing: Validating drowsiness detection accuracy
- Deployment: Running the system in real-time environment
- Maintenance: Enhancing features and fixing issues



The Waterfall model is suitable for this project as the requirements are clearly defined and the development process follows a structured and systematic approach.

SYSTEM DESIGN

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

System Design is the phase of software development in which the requirements identified during system analysis are transformed into a structured blueprint for system implementation. It defines the system architecture, processing flow, data organization, and interaction between system components to ensure reliable and efficient operation.

In the **AI-Based Drowsiness Detection System**, system design plays a crucial role as the application performs real-time video processing, facial feature extraction, drowsiness analysis, distraction detection, and alert generation. The system continuously monitors the driver's facial behavior using a webcam and applies computer vision and machine learning techniques to detect fatigue-related patterns such as prolonged eye closure, yawning, and lack of focus.

A well-designed architecture ensures real-time performance, accuracy of detection, minimal system latency, and smooth interaction between hardware (camera) and software components. The design phase focuses on defining the internal working of detection modules, alert mechanisms, data flow, and user interaction.

The system design of the AI-Based Drowsiness Detection System includes:

- Designing a lightweight data structure for storing session statistics such as drowsiness count
- Developing Data Flow Diagrams (DFDs) to represent data movement between system components
- Creating UML diagrams such as Use Case, Activity, Sequence, Class, and State Machine diagrams
- Designing modular components for detection, alerting, and visualization
- Defining clear input and output mechanisms for real-time monitoring

This structured system design improves reliability, scalability, and ease of future enhancements.

3.2 DATABASE DESIGN

The AI-Based Drowsiness Detection System is primarily a **real-time processing system** and does not require a traditional persistent database for its core operation. However, for analytical and future extension purposes, the system can maintain **in-memory or lightweight storage structures** to track session-level information such as drowsiness count, alert history, and distraction events.

Instead of a full-fledged database server, the system relies on:

- Runtime variables for real-time processing
- Optional file-based logging for session data
- Extendable design to support database integration if required

This design choice keeps the system lightweight, fast, and suitable for real-time deployment

3.2.1 ENTITY RELATIONSHIP MODEL

The Entity Relationship (ER) Model defines the logical structure of the **AI-Based Drowsiness Detection System** by representing its core entities, attributes, and the relationships between them. It provides a clear and structured view of how data is generated, processed, and utilized across system modules such as video capture, facial analysis, drowsiness detection, alert generation, and session monitoring. This model ensures data consistency, system reliability, and effective monitoring throughout the drowsiness detection process.

1. Entities

The primary entities identified in the AI-Based Drowsiness Detection System are **Driver, Camera, Detection_System, Alert, and Session_Data**.

- **Driver** represents the person whose alertness is continuously monitored by the system.
- **Camera** captures real-time facial video input of the driver.
- **Detection_System** performs facial landmark detection and computes drowsiness- related metrics.
- **Alert** stores information related to audio, visual, and voice warnings triggered by the system.
- **Session_Data** records runtime monitoring details such as drowsiness count and timestamps.

2. Attributes

Each entity contains attributes that define its specific properties.

- **Driver:**
  driver_id, name, monitoring_status, focus_status
- **Camera:**
  camera_id, resolution, frame_rate, status

- **Detection_System:**
  detection_id, EAR_value, MAR_value, distraction_score
- **Alert:**
  alert_id, alert_type, alert_status, triggered_time
- **Session_Data:**
  session_id, driver_id, drowsiness_count, alert_triggered, timestamp

3. Primary Key

Each entity includes a primary key (PK) to uniquely identify records:

- **driver_id** uniquely identifies a driver
- **camera_id** uniquely identifies the camera device
- **detection_id** uniquely identifies the detection engine instance
- **alert_id** uniquely identifies an alert
- **session_id** uniquely identifies a monitoring session

4. Foreign Key

Relationships between entities are defined using foreign keys:

- **driver_id** in Session_Data references Driver(driver_id)
- **session_id** in Alert references Session_Data(session_id)
- **camera_id** in Detection_System references Camera(camera_id)

5. Relationships

The ER model establishes the following relationships among entities:

- One **Driver** is monitored during multiple sessions (1:N).
- One **Camera** captures video for one driver at a time (1:1).
- One **Detection_System** processes multiple session records (1:N).
- One **Session_Data** entry may generate multiple alerts (1:N).
- One **Alert** is always associated with exactly one session (N:1).

6. Cardinality

Cardinality defines the number of associations between entities:

- One driver → Many session records (1:N)
- One camera → One driver (1:1)

- One detection system → Many session data records (1:N)
- One session → Many alerts (1:N)
- One alert → One session (N:1)

7.Generalisation/Specialisation

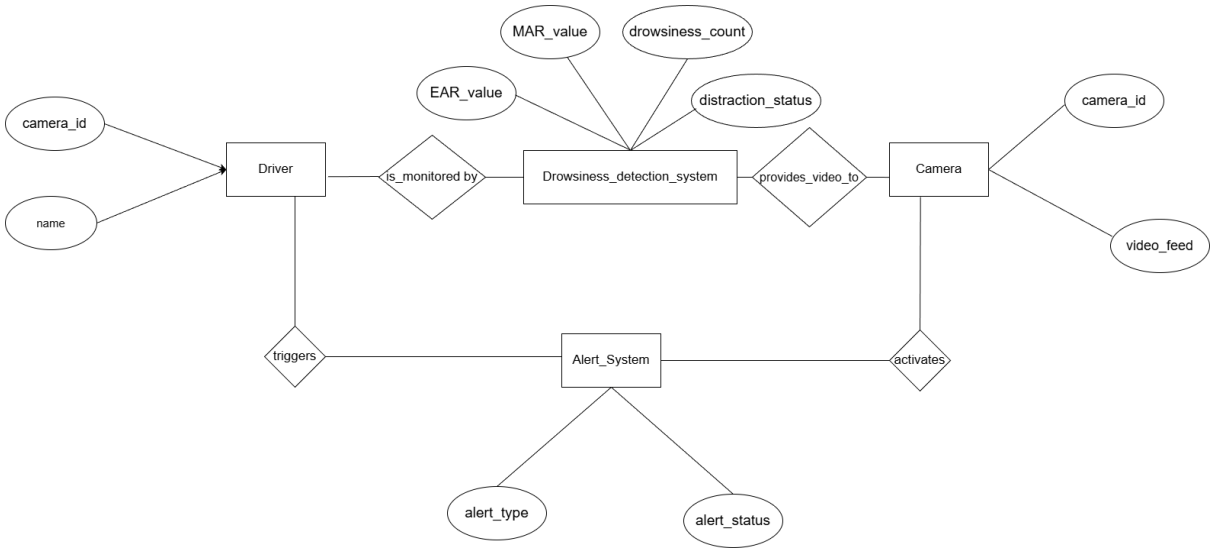
The **Driver** entity acts as the primary and specialized user entity in the AI-Based Drowsiness Detection System.Since the system focuses solely on driver monitoring, no further specialization is required. However, the design allows future extension where the Driver entity may be generalized into broader user categories if required.This structure supports modular design and ensures efficient monitoring without redundant user roles.

8.Connectivity

Connectivity defines how entities are logically linked within the system:

- Each session must be connected to one driver.
- Each detection process must receive input from one camera.
- Each alert must be linked to a specific monitoring session.
- Session data continuously connects the driver, detection system, and alert mechanism.

The ER model of the AI-Based Drowsiness Detection System accurately represents the interaction between monitoring components, detection logic, and alert generation. It ensures referential integrity, supports real-time processing, and forms the foundation for the system's data handling architecture.



3.2.2 DATA DICTIONARY

The data dictionary describes the key data elements used in the AI-Based Drowsiness Detection System during execution.

DRIVER

Field Name	Data Type	Description
driver_id	Integer (PK)	Unique identifier for the driver
name	Varchar	Name of the driver
monitoring_status	Boolean	Indicates if monitoring is active
focus_status	Varchar	Current focus state (Focused / Distracted)

CAMERA

Field Name	Data Type	Description
camera_id	Integer (PK)	Unique camera identifier
resolution	Varchar	Camera resolution
frame_rate	Integer	Frames per second
status	Boolean	Camera active/inactive

DETECTION_SYSTEM

Field Name	Data Type	Description
detection_id	Integer (PK)	Unique detection process ID
EAR_value	Float	Eye Aspect Ratio
MAR_value	Float	Mouth Aspect Ratio
distraction_score	Float	Head pose / focus score
detection_time	Timestamp	Detection timestamp

SESSION_DATA

Field Name	Data Type	Description
session_id	Integer (PK)	Unique session identifier
driver_id	Integer (FK)	References DRIVER(driver_id)
drowsiness_count	Integer	Number of drowsiness events
session_start	Timestamp	Monitoring start time
session_end	Timestamp	Monitoring end time

ALERT

Field Name	Data Type	Description
alert_id	Integer (PK)	Unique alert identifier
session_id	Integer (FK)	References SESSION_DATA(session_id)
alert_type	Varchar	Sound / Voice / Visual
alert_status	Boolean	Triggered or not
alert_time	Timestamp	Time alert was triggered

DISTRACTION_EVENT

Field Name	Data Type	Description
distraction_id	Integer (PK)	Unique distraction event ID
session_id	Integer (FK)	References SESSION_DATA(session_id)
direction	Varchar	Left / Right / Down
duration	Integer	Time distracted (seconds)

YAWN_EVENT

Field Name	Data Type	Description
yawn_id	Integer (PK)	Unique yawn event identifier
session_id	Integer (FK)	References SESSION_DATA(session_id)
MAR_value	Float	Mouth Aspect Ratio
detected_time	Timestamp	Yawn detection time

3.3 PROCESS DESIGN – DATA FLOW DIAGRAMS

Data Flow Diagrams (DFDs) are used to represent how data flows within the **AI-Based Drowsiness Detection System**. A Data Flow Diagram is a graphical representation of the movement of data between external entities, internal processing modules, and data stores within an information system. It focuses on **what data is processed, where the data originates, how it is transformed, and where it is delivered**, rather than the control flow or timing of operations.

In the AI-Based Drowsiness Detection System, DFDs provide a clear visualization of how the **driver**, **camera**, **detection modules**, and **alert mechanisms** interact during real-time monitoring. The diagrams illustrate the flow of live video input from the

webcam, facial landmark extraction, computation of eye and mouth aspect ratios, detection of drowsiness, yawning, and distraction, and the generation of visual, audio, and voice alerts.

The DFDs help in understanding the **system boundaries**, the decomposition of complex detection processes into simpler functional units, and the interaction between various components such as video capture, facial analysis, drowsiness evaluation, and alert generation. They also depict how session-related data such as drowsiness count and alert status is maintained during system execution.

Data Flow Diagrams are particularly useful during the design phase of the AI-Based Drowsiness Detection System as they provide a **high-level overview of real-time data processing**, which can later be expanded into more detailed representations. These diagrams

ensure clarity in system design, improve maintainability, and support future enhancements such as data logging, analytics, and database integration.

Data Flows

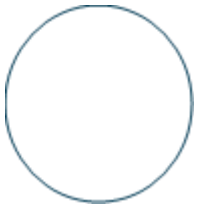
Data flows represent the movement of data from one component of the system to another and are depicted using arrows. Each arrow indicates the direction of data flow and is labeled with the type of data being transferred, such as ride details, user credentials, status updates, or alerts.

Basic data flow diagram symbols are: -

- A rectangle defines a source or destination of the data.

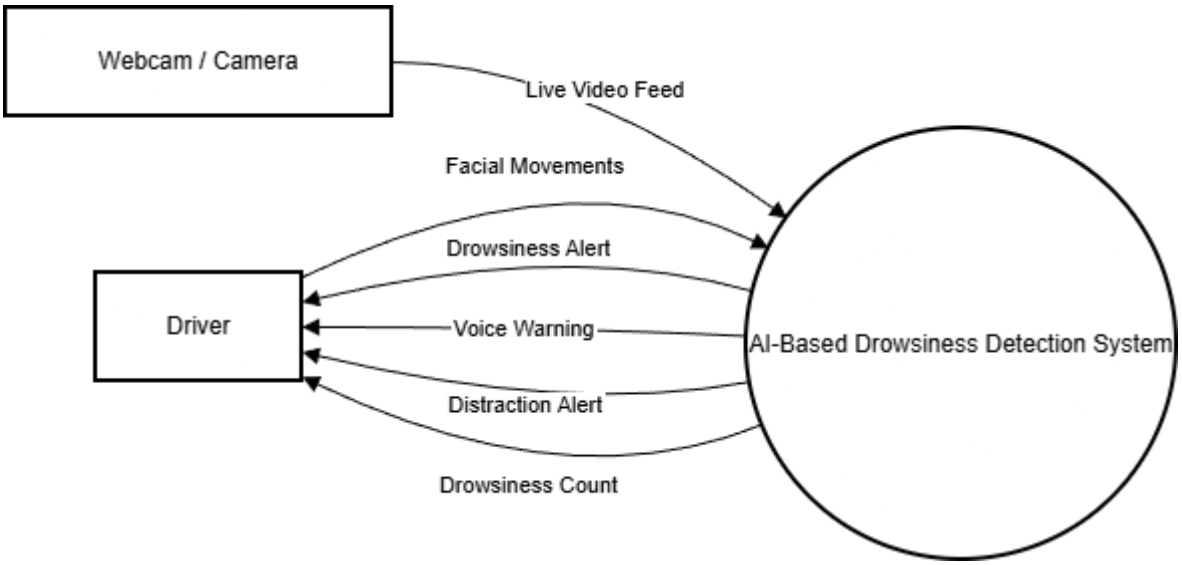
- An arrow identifies data flow, data in motion. It is a pipeline through which information flows

- A circle or bubble represents a process at transforms incoming data flows into outgoing dataflows.

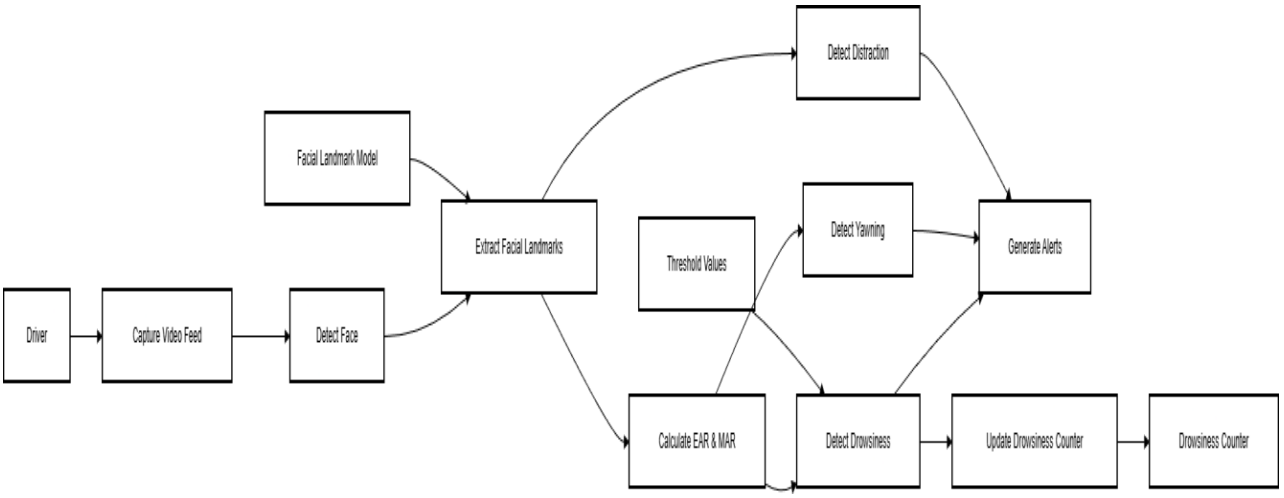


- An open rectangle is a data store

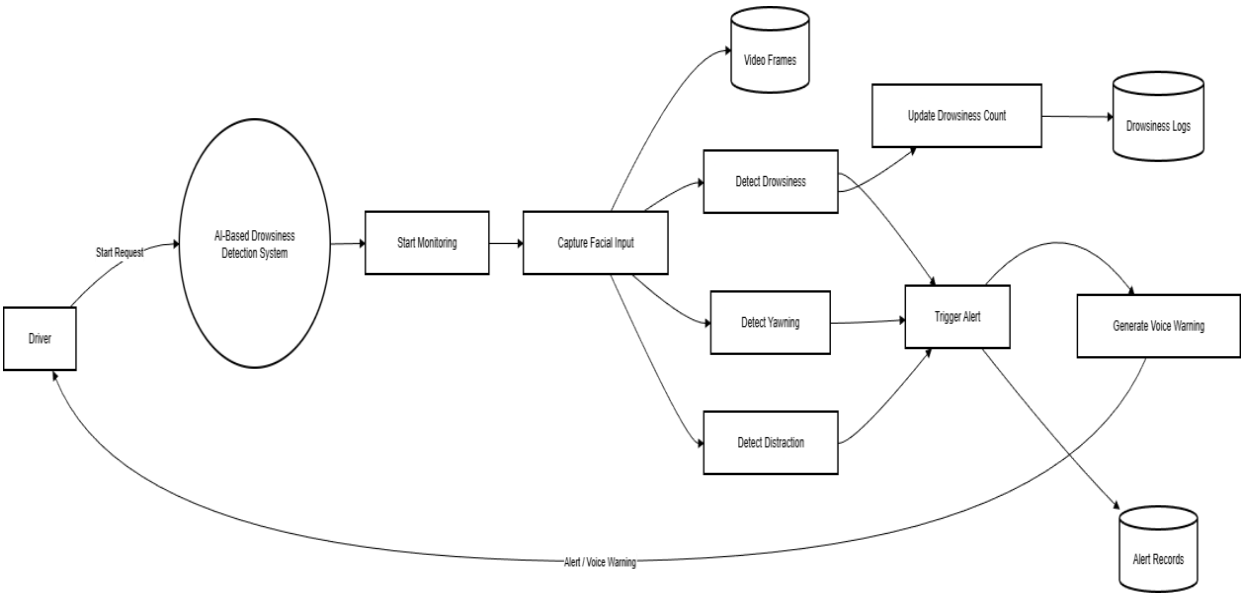
LEVEL 0 DFD



LEVEL 1 DFD-DRIVER



LEVEL 1 MONITORED USER



3.4 OBJECT-ORIENTED DESIGN – UML DIAGRAMS

Object-Oriented Design (OOD) structures the **AI-Based Drowsiness Detection System** around objects that represent real-world entities involved in driver monitoring and alert generation. Each object encapsulates both data and behavior, enabling modularity, reusability, maintainability, and scalability of the system.

The system is designed using object-oriented principles such as abstraction,

encapsulation, and modular decomposition. These principles help in clearly separating responsibilities among different components such as video capture, facial analysis, drowsiness detection, distraction detection, and alert generation.

Unified Modeling Language (UML) diagrams are used to visually represent both the **structural** and **behavioral** aspects of the system:

- **Structural diagrams**, such as the **Class Diagram**, describe system classes, their attributes, methods, and relationships.
- **Behavioral diagrams**, such as **Use Case, Activity, Sequence, and State Machine diagrams**, represent system workflows, interactions, and dynamic behavior during execution.

Together, these UML diagrams provide a comprehensive understanding of the architecture, design logic, and functional behavior of the AI-Based Drowsiness Detection System

3.4.1 DEPLOYMENT DIAGRAM

The Deployment Diagram represents the **physical architecture** of the **AI-Based Drowsiness Detection System** and illustrates how software components are deployed on hardware devices during runtime. It shows the execution environment of the system and how the driver interacts with the application using input and output devices.

In the AI-Based Drowsiness Detection System, the deployment architecture follows a **standalone, real-time desktop-based model**, where all processing is performed locally on a single computing device. This design eliminates dependency on external servers and ensures low latency, making it suitable for real-time driver monitoring.

The **Driver Device**, represented as a UML «device», consists of a personal computer or laptop equipped with a webcam and audio output system. The webcam continuously captures live video frames of the driver's face, which serve as the primary input to the system. The display and speakers are used to present visual alerts, alarm sounds, and voice warnings to the driver.

The **Processing Unit**, represented as a UML «node», hosts the core application developed using Python. This node runs the AI-based detection logic and includes the following internal software components:

- **Video Capture Module**, responsible for acquiring real-time video frames from the webcam using OpenCV.
- **Face Detection and Landmark Module**, implemented using dlib, which detects facial landmarks required for analysis.
- **Drowsiness Detection Module**, which computes Eye Aspect Ratio (EAR) and

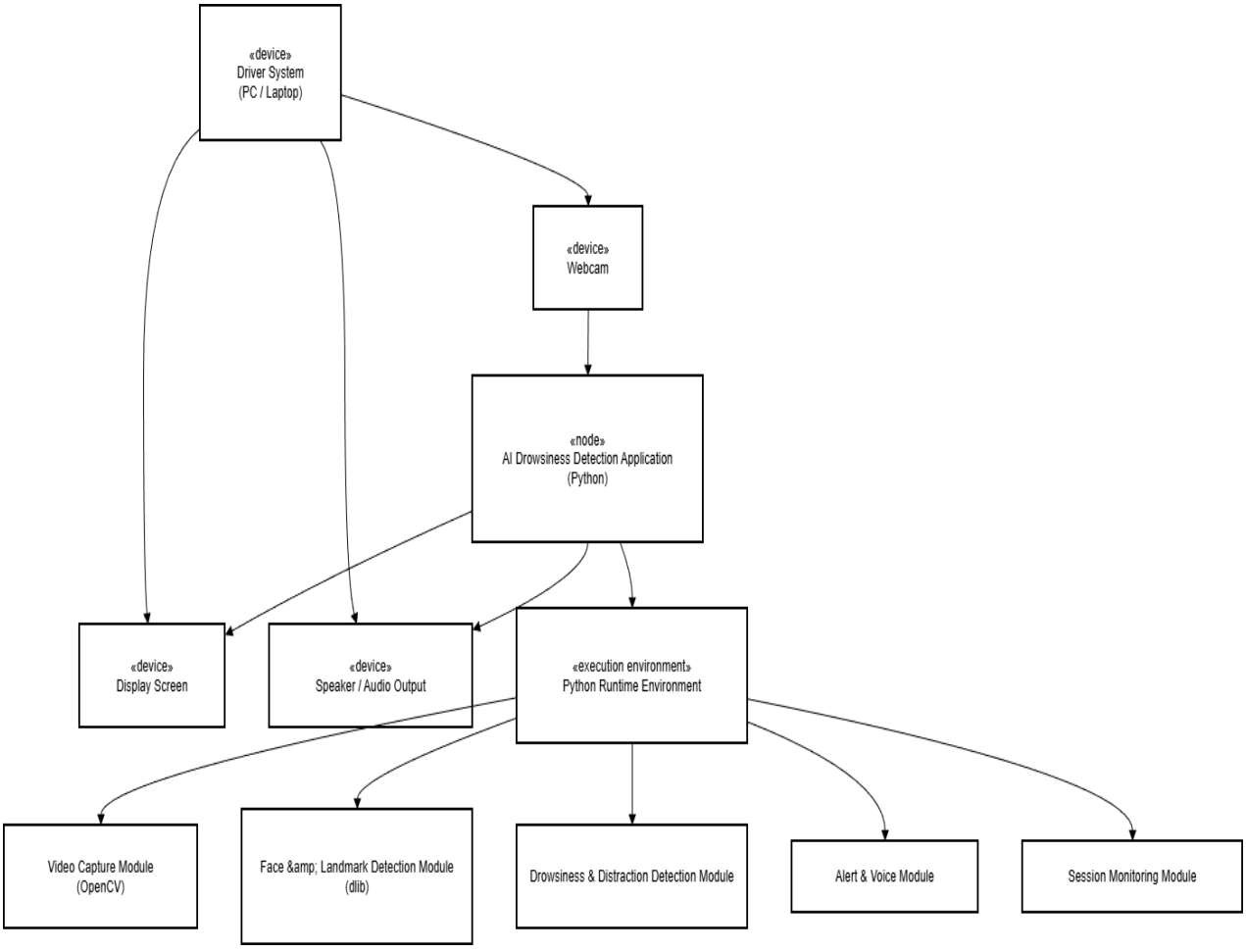
Mouth Aspect Ratio (MAR) to identify fatigue and yawning.

- **Distraction Detection Module**, which analyzes head orientation to detect loss of focus.
- **Alert Management Module**, which generates audio alarms, voice alerts, and on-screen warnings.
- **Session Monitoring Module**, which maintains runtime data such as drowsiness count and alert status.

All these modules operate within the same runtime environment and communicate internally to ensure seamless and efficient system operation.

The **Runtime Environment**, represented as a UML «execution environment», includes the Python interpreter along with supporting libraries such as OpenCV, dlib, NumPy, SciPy, and text-to-speech libraries. The operating system manages hardware access and process execution.

Overall, the deployment diagram clearly demonstrates how the AI-Based Drowsiness Detection System is physically structured and executed in a real-world environment. It highlights the integration of hardware components (camera, display, speakers) with software modules, ensuring real-time performance, reliability, and ease of deployment without requiring external servers or network connectivity.



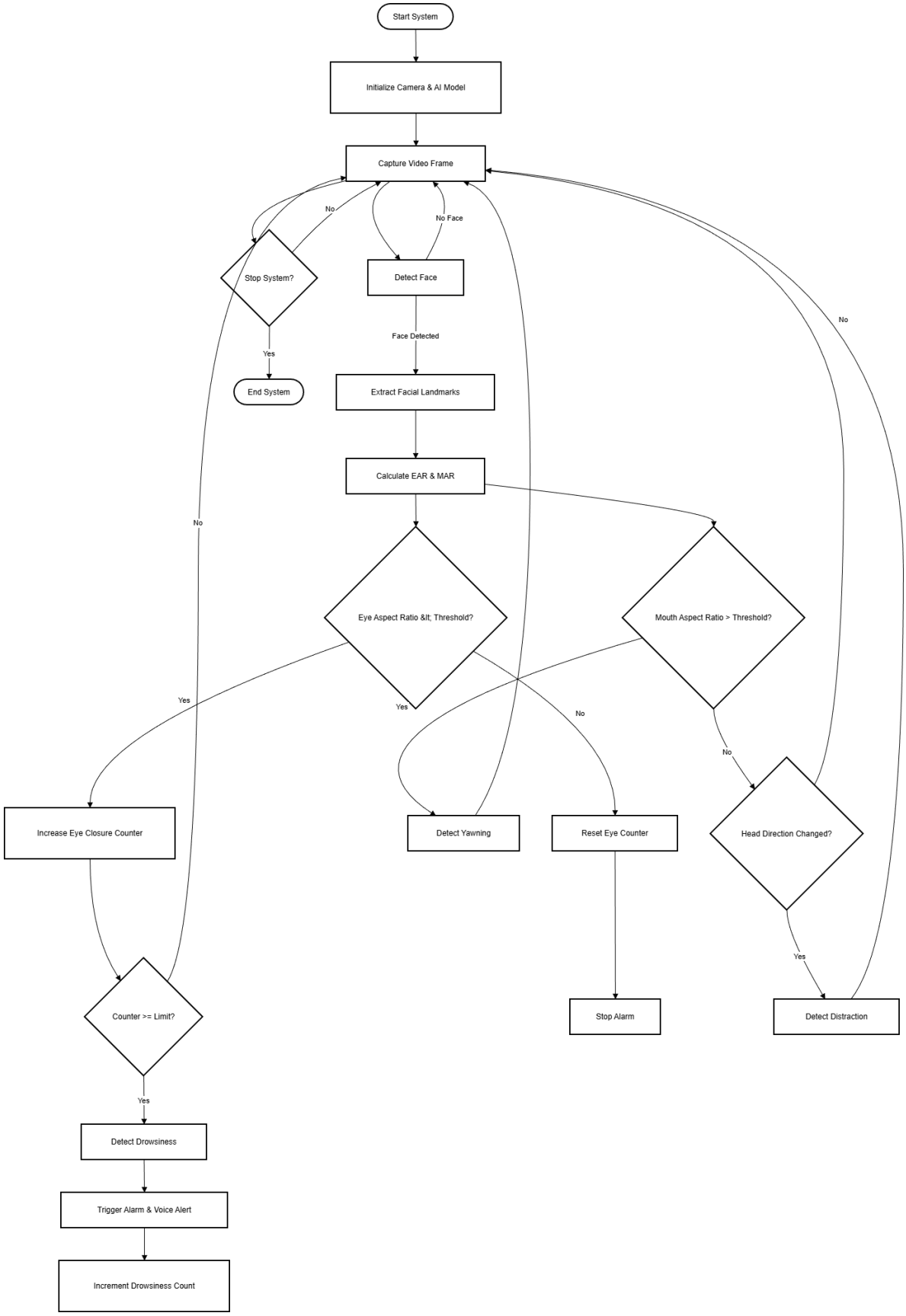
3.4.2 ACTIVITY DIAGRAM

The Activity Diagram is a behavioral UML diagram that represents the flow of activities and control within the **AI-Based Drowsiness Detection System**. It illustrates how different actions are performed sequentially and conditionally during the real-time monitoring of the driver. The activity diagram models the end-to-end workflow starting from system initialization, camera activation, video frame capture, facial feature analysis, and drowsiness evaluation.

In the AI-Based Drowsiness Detection System, the activity diagram captures the continuous cycle of monitoring in which the system detects the driver's face, extracts facial landmarks, and calculates Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR). Based on these computed values, the system determines whether the driver is in a normal, drowsy, yawning, or distracted state.

The diagram also represents important decision points such as whether a face is detected, whether eye closure exceeds the threshold, whether yawning is detected, and whether the driver is distracted. Conditional flows are used to trigger alerts such as alarm sounds and voice warnings when unsafe conditions are identified. Once the driver regains alertness, the system resets the alert state and continues monitoring.

The activity diagram helps in understanding the operational flow of the system, identifying critical decision paths, and validating that the implemented real-time monitoring logic aligns with the intended system behavior. It also ensures clarity in how continuous monitoring, alert generation, and system termination are handled within the AI-Based Drowsiness Detection System.


Start System
Initialize Camera & AI Model
Capture Video Frame
No
No Face
Stop System?
Detect Face
Face Detected
Yes
End System
Extract Facial Landmarks
Calculate EAR & MAR
No
Eye Aspect Ratio < Threshold?
Mouth Aspect Ratio > Threshold?
Yes
Yes
No
No
Increase Eye Closure Counter
Detect Yawning
Reset Eye Counter
Head Direction Changed?
Yes
Counter >= Limit?
Stop Alarm
Detect Distraction
Yes
Detect Drowsiness
Trigger Alarm & Voice Alert
Increment Drowsiness Count

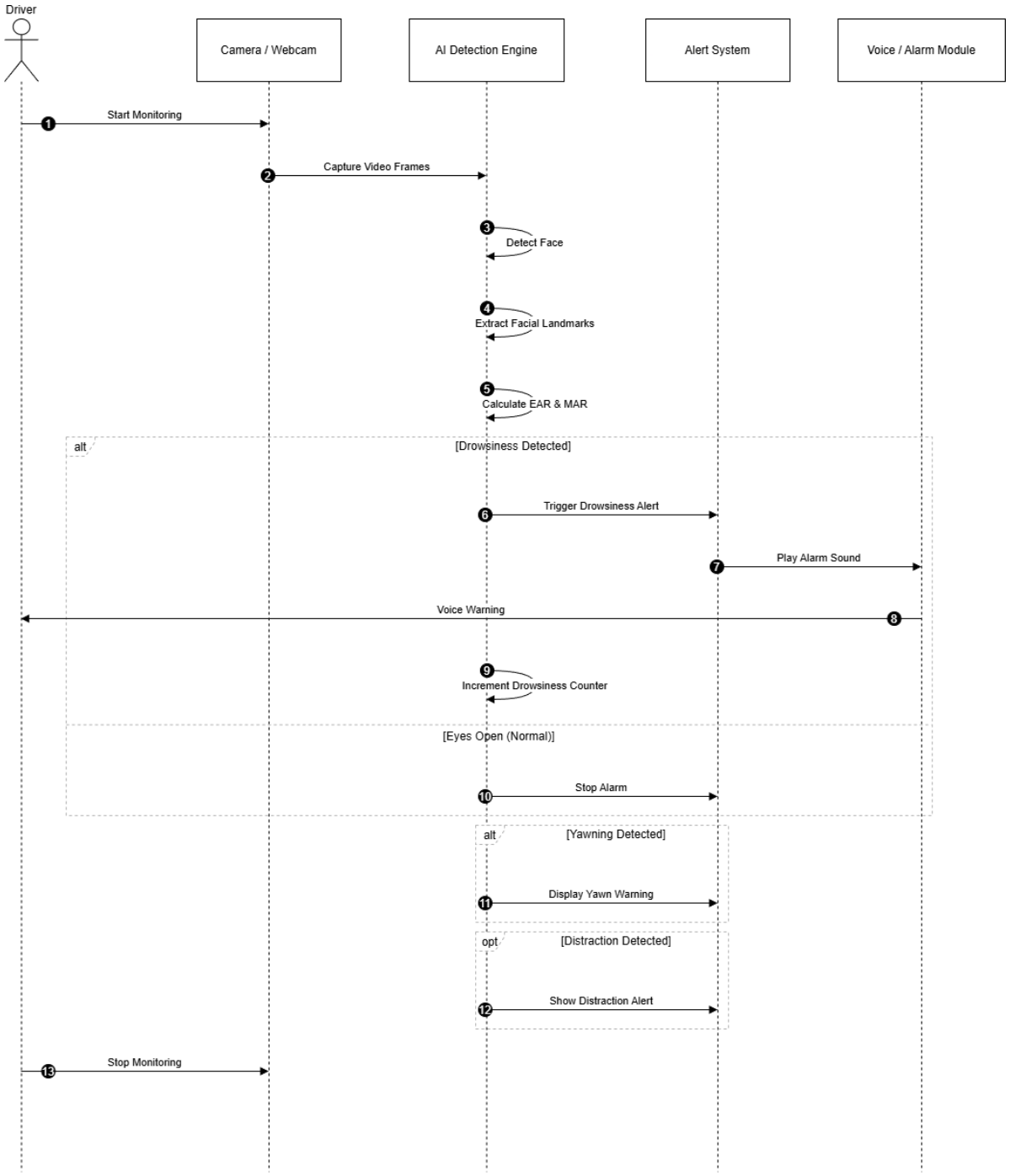
3.4.3 SEQUENCE DIAGRAM

The Sequence Diagram is a UML interaction diagram that represents the **time-ordered communication** between system components and users. It focuses on how objects interact with one another through message exchanges to accomplish a specific operation, emphasizing the sequence in which these interactions occur.

In the **AI-Based Drowsiness Detection System**, the sequence diagram illustrates the interaction between the **Driver**, **Camera**, **AI Detection Engine**, and **Alert System** during real- time monitoring. The diagram clearly depicts the chronological flow of events starting from the driver initiating the monitoring process, followed by continuous video frame capture through the camera. These frames are then processed by the AI detection engine to perform face detection, facial landmark extraction, and computation of Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR).

Based on the computed values, the system evaluates the driver's alertness level and determines whether drowsiness, yawning, or distraction is present. When unsafe conditions are detected, the AI engine communicates with the alert system to trigger appropriate responses such as alarm sounds, voice warnings, and on-screen notifications. The sequence diagram also represents the system's behavior when the driver regains alertness, showing how alerts are stopped and monitoring continues.

By visualizing message flow over time, the sequence diagram ensures proper synchronization between system components, validates the interaction logic of the real-time detection process, and confirms that alerts are generated accurately in response to the driver's state. This diagram helps in verifying that the implemented system behavior aligns with the intended design of the AI-Based Drowsiness Detection System.

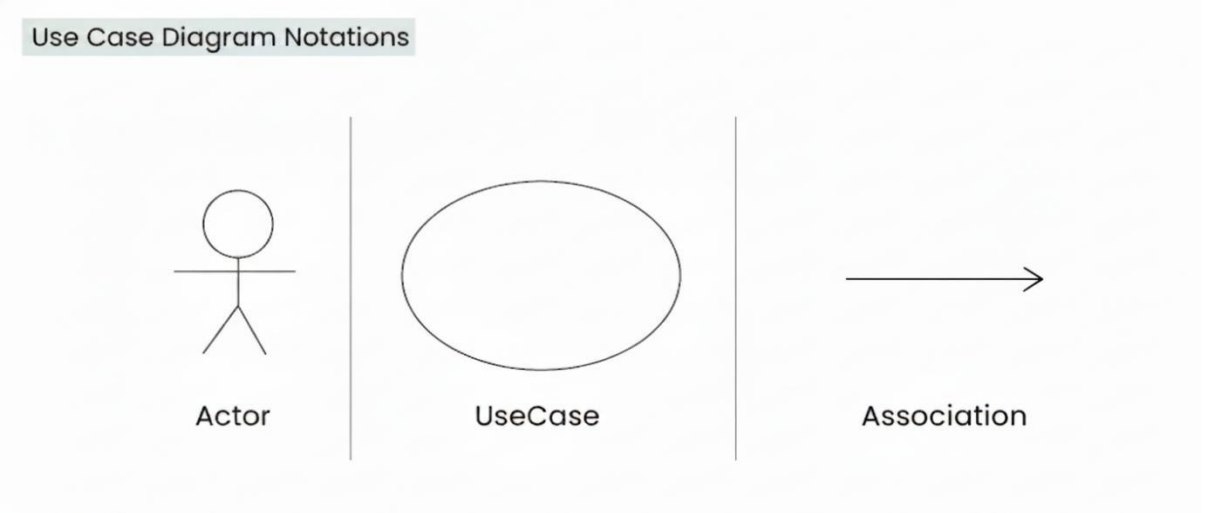

Driver
Camera / Webcam
AI Detection Engine
Alert System
Voice / Alarm Module
1 Start Monitoring
2 Capture Video Frames
3 Detect Face
4 Extract Facial Landmarks
5 Calculate EAR & MAR
alt
[Drowsiness Detected]
6 Trigger Drowsiness Alert
7 Play Alarm Sound
8 Voice Warning
9 Increment Drowsiness Counter
[Eyes Open (Normal)]
10 Stop Alarm
alt
[Yawning Detected]
11 Display Yawn Warning
opt
[Distraction Detected]
12 Show Distraction Alert
13 Stop Monitoring

3.4.4 USE CASE DIAGRAM

The Use Case Diagram provides a functional overview of the **AI-Based Drowsiness Detection System** from the perspective of external users (actors). It focuses on **what the system does** rather than how the internal processing is implemented. The use case diagram helps in identifying the key functionalities offered by the system and how the user interacts with them.

In the AI-Based Drowsiness Detection System, the primary actor is the **Driver**, who interacts with the system during real-time monitoring. The driver initiates the monitoring process, provides facial input through the camera, and receives alerts when drowsiness, yawning, or distraction is detected. The system continuously processes facial features and generates visual, audio, and voice warnings to ensure driver safety.

The use case diagram clearly defines the functional requirements of the system such as starting and stopping the monitoring process, detecting driver fatigue, generating alerts, and displaying monitoring information like the drowsiness counter. By representing these interactions, the diagram ensures that all functional requirements of the AI-Based Drowsiness Detection System are clearly captured and aligned with user expectations.



Actors Involved

3.4.4.1 Driver:

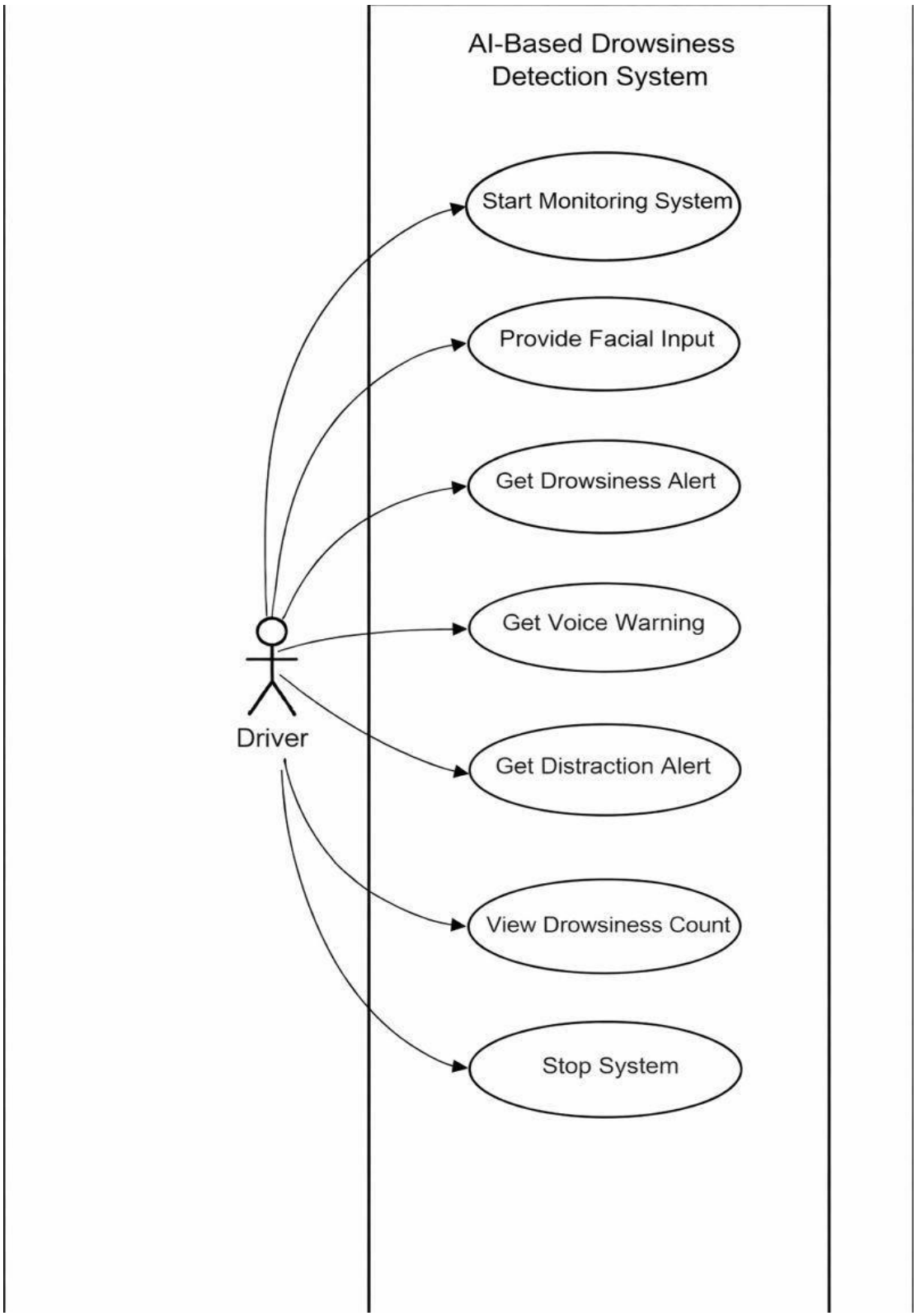
The primary user of the system whose facial expressions and alertness levels are continuously monitored.

3.4.4.2 Use Case:

Shown as an oval, it represents the functions or features that the system performs.

3.4.4.3 Association:

Depicted by a straight line connecting actors to use cases, showing their interaction or communication.


AI-Based Drowsiness Detection System
Start Monitoring System
Provide Facial Input
Get Drowsiness Alert
Get Voice Warning
Get Distraction Alert
View Drowsiness Count
Stop System
Driver

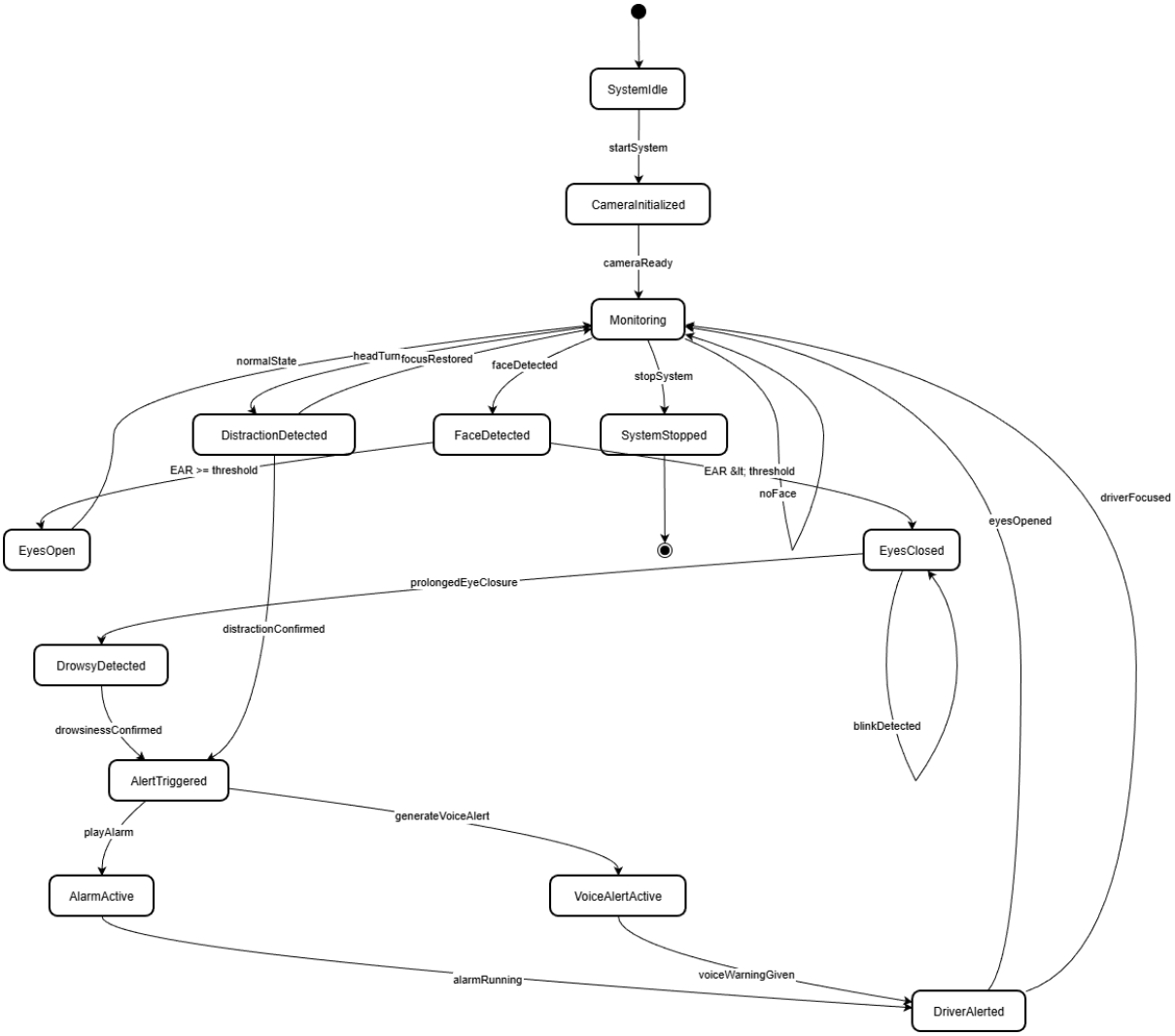
3.4.5 STATE MACHINE DIAGRAM

The State Machine Diagram models the lifecycle of the **monitoring and alerting process** in the **AI-Based Drowsiness Detection System** by depicting its possible states and the valid transitions between them. It focuses on how the system dynamically changes its behavior in response to real-time events such as facial detection results, eye closure duration, yawning, distraction, and user actions.

In the AI-Based Drowsiness Detection System, the state machine diagram captures system states such as **Idle**, **Camera Initialized**, **Monitoring**, **Normal State**, **Drowsiness Detected**, **Distraction Detected**, **Alert Active**, and **System Stopped**. Transitions between these states occur based on events such as system start, face detection, prolonged eye closure, head movement indicating distraction, alert activation, and recovery of driver alertness.

For example, when the system starts, it transitions from the *Idle* state to the *Monitoring* state after camera initialization. If the driver's eyes remain closed beyond a predefined threshold, the system transitions to the *Drowsiness Detected* state and activates alerts. Once the driver opens their eyes or regains focus, the system transitions back to the *Monitoring* state. If the system is manually stopped, it enters the *System Stopped* state.

This diagram ensures that all state transitions follow well-defined rules, preventing invalid or inconsistent system behavior. It also helps in validating that the real-time monitoring logic, alert generation, and recovery mechanisms are correctly implemented, thereby maintaining reliability and safety throughout the operation of the AI-Based Drowsiness Detection System.



3.4.6 CLASS DIAGRAM

The Class Diagram is a structural UML diagram that represents the **static structure** of the **AI- Based Drowsiness Detection System**. It defines the system's core classes, their attributes, methods, and the relationships among them. This diagram focuses on how data and functionality are organized within the system and how different components interact at a structural level.

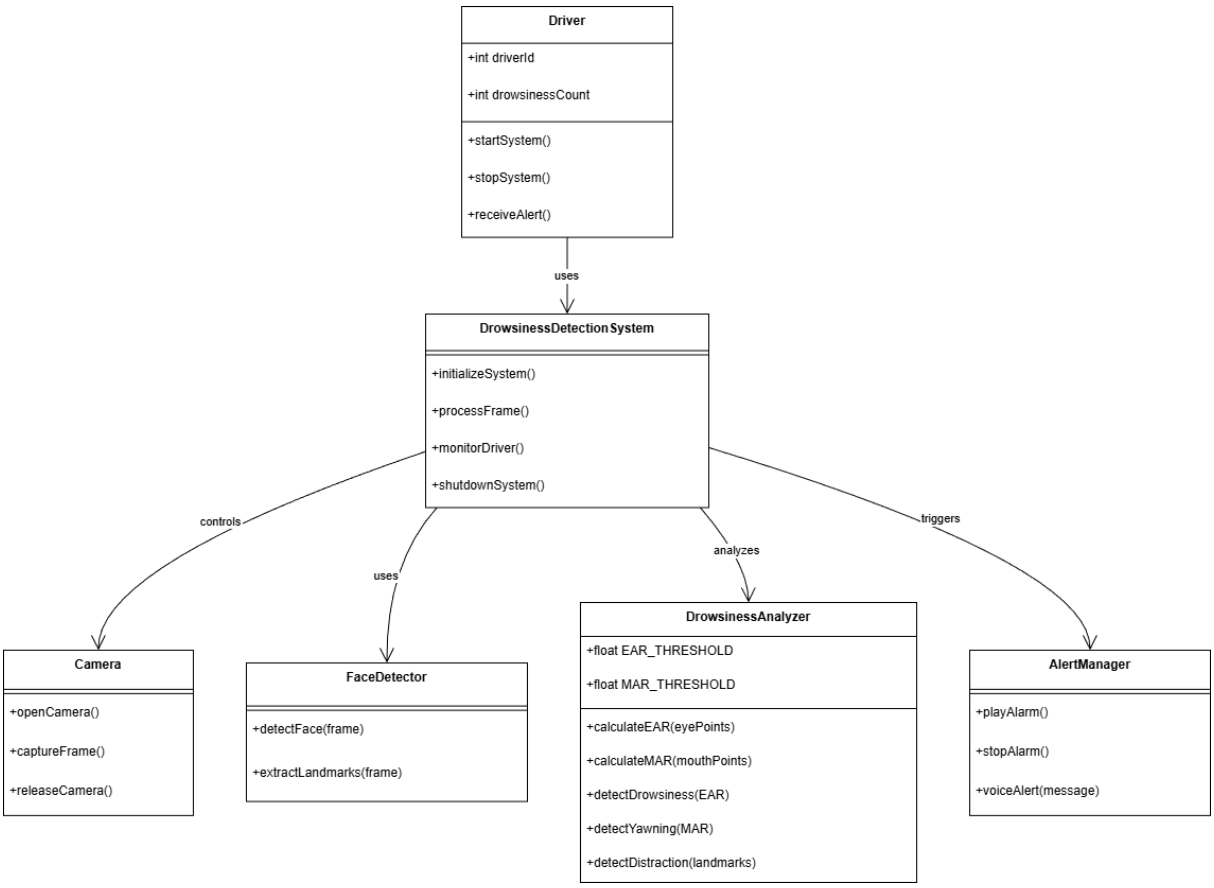
In the AI-Based Drowsiness Detection System, the class diagram includes classes such as **Driver**, **Camera**, **DrowsinessDetectionSystem**, **FaceDetector**, **DrowsinessAnalyzer**, **AlertManager**, and **SessionData**. These classes collectively represent the major functional units involved in real-time video capture, facial feature analysis, drowsiness and distraction detection, alert generation, and session monitoring.

The **Driver** class represents the user being monitored by the system. The **Camera** class handles real-time video input from the webcam. The **FaceDetector** class is responsible for detecting facial landmarks from captured frames. The **DrowsinessAnalyzer** class

performs computations such as Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and distraction analysis. The **AlertManager** class manages audio alarms, voice alerts, and visual warnings. The **SessionData** class maintains runtime information such as drowsiness count and alert

status. The **DrowsinessDetectionSystem** class acts as the central controller that coordinates interactions among all other classes.

The class diagram illustrates relationships such as association and dependency between these classes, showing how the main detection system uses supporting modules to perform its operations. This diagram closely reflects the logical structure of the implemented Python-based system and helps developers and evaluators understand object responsibilities, data flow, and overall system architecture.



3.5 MODULAR DESIGN

Modular design is a software design approach that divides the entire system into smaller, independent, and manageable units called **modules**. Each module is responsible for performing a specific function and communicates with other modules through well-defined interfaces. This design approach enhances system clarity, improves

maintainability, supports scalability, and simplifies debugging and future enhancements.

In the **AI-Based Drowsiness Detection System**, modular design is adopted to efficiently manage real-time video processing and fatigue detection tasks. The system performs continuous monitoring, facial feature analysis, alert generation, and session tracking, which can become complex if handled as a single unit. By dividing the system into dedicated modules such as video capture, facial analysis, drowsiness detection, distraction detection, and alert management, the system ensures **high cohesion within modules and loose coupling between modules**.

Each module operates independently while contributing to the overall functionality of the system. This modular approach makes the AI-Based Drowsiness Detection System easier to understand, test, modify, and extend, for example, by adding data logging, advanced analytics, or cloud integration in the future.

3.5.1 STRUCTURE CHART

A Structure Chart represents the **hierarchical decomposition** of the system into modules and sub-modules. It illustrates how control flows from the main system module to individual functional components and how these components interact to perform system operations.

In the **AI-Based Drowsiness Detection System**, the top-level module controls the overall execution of the system, including initialization, continuous monitoring, and system termination. The lower-level modules handle specific operations such as video acquisition, facial landmark detection, drowsiness analysis, distraction detection, and alert generation.

Main Modules of AI-Based Drowsiness Detection System

3.5.1.1 Main Control Module
3.5.1.2 Video Capture Module
3.5.1.3 Facial Detection & Landmark Module
3.5.1.4 Drowsiness Detection Module
3.5.1.5 Distraction Detection Module
3.5.1.6 Alert Management Module
3.5.1.7 Session Monitoring Module
3.5.1.8 Display Module

The structure chart illustrates how the **Main Control Module** coordinates all other modules and ensures smooth execution of the real-time monitoring process. Each sub-module performs its designated task and returns results to the main module, which then

decides the next course of action based on detection outcomes.

This hierarchical organization improves system reliability and ensures that the AI-Based Drowsiness Detection System functions efficiently as a unified yet modular application.


AI Drowsiness Detection System
Video Capture Module
Access Webcam
Read Video Frames
Face Detection Module
Detect Face
Detect Facial Landmarks
Feature Extraction Module
Compute EAR
Compute MAR
Compute Head Direction
Drowsiness Detection Module
Detect Eye Closure
Update Drowsiness Counter
Distraction Detection Module
Detect Gaze Direction
Detect Loss of Focus
Alert Module
Play Alarm Sound
Generate Voice Alert
Display & Reporting Module
Display Alerts on Screen
Show Drowsiness Count

3.5.2 MODULE DESCRIPTIONS

Video Capture Module

This module is responsible for accessing the webcam and continuously capturing real-time video frames of the driver. It ensures smooth frame acquisition and provides visual input for further processing by downstream modules.

Face Detection Module

The Face Detection Module identifies the presence of a human face within each video frame using a pre-trained frontal face detector. Only frames containing a valid face are processed further, ensuring accuracy and reducing unnecessary computation.

Feature Extraction Module

This module extracts key facial landmarks such as eye and mouth regions using a landmark prediction model. It computes critical features like Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), which are essential indicators of drowsiness and yawning.

Drowsiness Detection Module

The Drowsiness Detection Module analyzes EAR values over consecutive frames to detect prolonged eye closure. If the EAR remains below a predefined threshold for a specified number of frames, the system identifies the driver as drowsy and increments the drowsiness counter.

Distraction Detection Module

This module detects driver distraction by analyzing facial orientation and gaze direction. If the driver is not looking straight ahead for a prolonged duration, the system flags distraction and generates an alert.

Alert Module

The Alert Module is responsible for triggering warning mechanisms when drowsiness or distraction is detected. It generates:

Continuous alarm sounds

Voice alerts to wake the driver

Alerts remain active until the driver regains normal alertness.

Display & Reporting Module

This module overlays visual information on the video feed, including drowsiness alerts, yawning detection, distraction warnings, and the drowsiness counter. It ensures real-time feedback to the driver.

Importance of Modular Design

- Improves system maintainability and clarity
- Allows independent testing of modules
- Supports future feature expansion (fatigue prediction, cloud logging)
- Reduces system complexity
- Enhances real-time performance and reliability

3.6 INPUT DESIGN

Input design defines how data enters the AI-Based Drowsiness Detection System. The system primarily relies on **real-time visual input** and **pre-trained models** rather than traditional form- based user input.

Input

- Captured through the system webcam
- Provides continuous facial images of the driver
- Serves as the core input for all detection processes

Facial Landmark Model Input

- Uses a pre-trained facial landmark predictor file
- Supplies reference points for eye, mouth, and face regions

System Configuration Inputs

- Threshold values for EAR and MAR
- Frame count for drowsiness detection
- Audio alert file paths

Input Design Features

- **Real-time Processing:** Continuous frame capture without user intervention
- **Validation:** Frames without detected faces are ignored
- **Accuracy:** Uses standardized facial landmarks

- **Minimal User Interaction:** Fully automated monitoring system
- **Error Handling:** Graceful handling of camera disconnection or missing model files

This input design ensures accurate, fast, and reliable data acquisition for real-time drowsiness monitoring.

3.7 OUTPUT DESIGN

Output design defines how system results are presented clearly and effectively to the driver.

Objectives of Output Design

- Provide immediate feedback to the driver
- Ensure high visibility of warnings
- Minimize driver distraction
- Support real-time decision-making

Visual Outputs

Live webcam feed with facial landmark
overlays On-screen messages:

- "DROWSINESS ALERT"
- "YAWNING DETECTED"
- "DISTRACTION DETECTED"

Display of EAR and MAR values.
Drowsiness counter visible on screen.

Audio Outputs

- Continuous alarm sound during drowsiness.
- Voice warning alert to wake the driver.
- Alerts stop automatically when alertness is restored.

Output Delivery Formats

- Real-Time Video Display
- Text Overlays
- Audio Alerts (Beep + Voice)

Benefits of Output Design

- Immediate driver awareness

- Prevents fatigue-related accidents
- Non-intrusive yet effective alerts
- Works without internet connectivity

SYSTEM ENVIRONMENT

CHAPTER 4

SYSTEM ENVIRONMENT

4.1 INTRODUCTION

The system environment defines the hardware and software setup required for the successful development and execution of the AI-Based Driver Drowsiness Detection System. It includes the tools, platforms, programming languages, and system configurations used to implement the project.

The system is developed using Python and various computer vision libraries, making it a lightweight and efficient solution that can run on standard computing devices without requiring specialized hardware.

4.2 SOFTWARE REQUIREMENTS SPECIFICATION

The software requirements for the system include:

- Operating System: Windows 10 or above
- Programming Language: Python
- Libraries and Packages:
- OpenCV (for video processing)
- dlib (for facial landmark detection)
- NumPy (for numerical computations)
- SciPy (for distance calculations)
- pyttsx3 (for voice alerts)
- winsound (for alarm sound)
- Development Environment:
- Anaconda / VS Code / Jupyter Notebook

These software components enable real-time processing, facial analysis, and alert generation.

4.3 HARDWARE REQUIREMENTS SPECIFICATION

The system requires minimal hardware components:

- Laptop or Desktop Computer
- Webcam (built-in or external)
- Minimum 4GB RAM
- Processor: Intel i3 or above

The system does not require any additional sensors or hardware devices, making it cost-effective and easy to deploy.

4.4 TOOLS AND PLATFORMS

The following tools and platforms were used in the development of the system:

- Anaconda: Used for managing Python environments and dependencies
- Visual Studio Code: Used for writing and editing code
- Python Libraries: Used for implementing system functionality
- GitHub: Used for version control and project backup

These tools provide a stable and efficient development environment.

4.5 FRONT-END TOOL

The front-end of the system is based on a graphical display generated using OpenCV.
Displays real-time video feed.
Shows alert messages such as:

- Drowsiness Alert
- Yawning Detected
- Distraction Alert
- Face Not Detected
- Risk Level Indicator
-

The interface is simple and user-friendly, allowing easy monitoring of driver behavior.

4.6 BACK-END TOOL

- The back-end of the system is implemented using Python.
- Processes video frames
- Detects facial features
- Calculates EAR and MAR values
- Implements detection algorithms
- Triggers alerts
- Generates session logs
- Python serves as the core engine of the system, handling all processing and decision-making operations.

4.7 OPERATING ENVIRONMENT

- The system operates in a real-time environment with continuous video input from the webcam.
- Real-time processing of video frames
- Continuous monitoring loop
- Immediate response to detected events

- File-based storage for session logs
- The system performs efficiently under normal lighting conditions and requires the driver’s face to be clearly visible.

SYSTEM IMPLEMENTATION

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

System implementation is the phase in which the designed system is developed, executed, and tested. It involves writing code, debugging errors, validating system functionality, and ensuring that all modules work together efficiently.

The AI-Based Driver Drowsiness Detection System is implemented using the Python programming language along with various computer vision libraries. The system captures real-time video input through a webcam and processes each frame continuously. Facial landmark detection is performed using dlib, while OpenCV is used for image processing.

Based on the extracted facial features, the system calculates Eye Aspect Ratio (EAR) to detect drowsiness and Mouth Aspect Ratio (MAR) to detect yawning. It also analyzes head movement to detect driver distraction. When abnormal behavior is detected, the system generates alerts using alarm sound and voice output.

Additionally, the system displays a real-time risk level indicator and generates a session summary report at the end of execution. The implementation is designed to be efficient, real-time, and independent of additional hardware.

5.2 CODING

The system is implemented using Python programming language, which is widely used for machine learning and computer vision applications. The code is organized into multiple sections to ensure clarity and proper execution. Initially, all required libraries such as OpenCV, dlib, NumPy, SciPy, pyttsx3, and winsound are imported to provide functionalities like video processing, facial landmark detection, numerical computation, voice alert, and sound generation.

Functions are defined for key operations such as calculating the Eye Aspect Ratio (EAR) for detecting eye closure and the Mouth Aspect Ratio (MAR) for detecting yawning. Additional functions are implemented to handle alert mechanisms, including playing alarm sounds and generating voice alerts using text-to-speech.

The system uses a webcam to capture real-time video input. Each frame is converted into grayscale for efficient processing, and face detection is performed using dlib's pre-trained model. Facial landmarks are then extracted, and relevant points are used to compute EAR and MAR values. Based on predefined threshold values, the system determines whether the driver is drowsy, yawning, or distracted.

Conditional statements are used to trigger alerts when abnormal behavior is detected for a certain number of consecutive frames. The system also maintains counters for drowsiness,

yawning, and distraction events. A risk level indicator is updated dynamically based on these events.

Finally, when the system is stopped, a session summary is generated and stored in a text file, providing details such as total events and session duration. The coding structure ensures real-time performance, accuracy, and ease of understanding.

5.3 CODING STANDARDS

The coding standards followed in this project ensure that the code is clean, readable, and maintainable. Proper indentation and formatting are used throughout the program to improve clarity and avoid syntax errors. Meaningful variable names and function names are chosen to clearly represent their purpose, making the code easier to understand.

The program is structured in a modular manner, where different functionalities such as detection, alert generation, and logging are separated into distinct sections. Comments are included in the code to explain important logic and improve readability. Reusable functions are created for common operations such as calculating Eye Aspect Ratio and Mouth Aspect Ratio.

Error handling and debugging practices are followed to ensure stable execution. Overall, the coding standards help in maintaining consistency, reducing complexity, and improving the overall quality of the system.

5.4 SAMPLE CODES

Main Code

```
import cv2

import dlib

import numpy as np

from scipy.spatial import distance as dist

import winsound

import os

import pyttsx3

import threading

import time

from datetime import datetime
```

VOICE ENGINE

```
engine = pyttsx3.init()
```

```
engine.setProperty("rate", 160)
engine.setProperty("volume", 1.0)
```

FUNCTIONS

```
def eye_aspect_ratio(eye):
A = dist.euclidean(eye[1], eye[5])
B = dist.euclidean(eye[2], eye[4])
C = dist.euclidean(eye[0], eye[3])
return (A + B) / (2.0 * C)

def mouth_aspect_ratio(mouth):
A = dist.euclidean(mouth[2], mouth[10])
B = dist.euclidean(mouth[4], mouth[8])
C = dist.euclidean(mouth[0], mouth[6])
return (A + B) / (2.0 * C)

def start_alarm():
sound_path = os.path.join("assets", "alarm.wav")
winsound.PlaySound(
sound_path,
winsound.SND_FILENAME | winsound.SND_ASYNC | winsound.SND_LOOP
)

def stop_alarm():
winsound.PlaySound(None, winsound.SND_PURGE)

def speak_alert(text):
def run_voice():
engine.say(text)
```

```
engine.runAndWait()
threading.Thread(target=run_voice, daemon=True).start()
```

CONSTANTS

```
EYE_THRESH = 0.25
EYE_FRAMES = 10
MOUTH_THRESH = 0.75
YAWN_FRAMES = 15
DISTRACTION_THRESH = 12
DISTRACTION_FRAMES = 12
```

COUNTERS

```
eye_counter = 0
alarm_on = False
voice_on = False
drowsy_count = 0
distract_counter = 0
distraction_count = 0
yawn_counter = 0
yawn_count = 0
session_start = time.time()
```

LOAD MODELS

```
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor(
"models/shape_predictor_68_face_landmarks.dat"
)

LEFT_EYE = list(range(42, 48))
RIGHT_EYE = list(range(36, 42))
```

```
MOUTH = list(range(48, 68))
NOSE_POINT = 30
```

WEBCAM

```
cap = cv2.VideoCapture(0)
print("[INFO] Drowsiness + Distraction Detection Started")

while True:
ret, frame = cap.read()
if not ret:
break

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = detector(gray)
```

FACE NOT DETECTED

```
if len(faces) == 0:
cv2.putText(frame, "FACE NOT DETECTED",
(30, 170), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (0, 0, 255), 2)

for face in faces:
shape = predictor(gray, face)
coords = np.array([[p.x, p.y] for p in shape.parts()])

leftEAR = eye_aspect_ratio(coords[LEFT_EYE])
rightEAR = eye_aspect_ratio(coords[RIGHT_EYE])
ear = (leftEAR + rightEAR) / 2.0
mar = mouth_aspect_ratio(coords[MOUTH])
```

```
cv2.drawContours(frame, [cv2.convexHull(coords[LEFT_EYE])], -1, (0, 255, 0), 1)
cv2.drawContours(frame, [cv2.convexHull(coords[RIGHT_EYE])], -1, (0, 255, 0), 1)
cv2.drawContours(frame, [cv2.convexHull(coords[MOUTH])], -1, (255, 0, 0), 1)
```

DROWSINESS

```
if ear < EYE_THRESH:
eye_counter += 1
if eye_counter >= EYE_FRAMES:
cv2.putText(frame, "DROWSINESS ALERT",
(30, 50), cv2.FONT_HERSHEY_SIMPLEX,
1, (0, 0, 255), 3)

if not alarm_on:
alarm_on = True
start_alarm()
drowsy_count += 1
if not voice_on:
voice_on = True
speak_alert("Wake up. You are feeling drowsy.")
else:
eye_counter = 0
if alarm_on:
stop_alarm()
alarm_on = False
voice_on = False
```

DISTRACTION

```
left_eye_center = np.mean(coords[LEFT_EYE], axis=0)
```

```
right_eye_center = np.mean(coords[RIGHT_EYE], axis=0)
eye_center_x = int((left_eye_center[0] + right_eye_center[0]) / 2)
nose_x = coords[NOSE_POINT][0]

if abs(nose_x - eye_center_x) > DISTRACTION_THRESH:
distract_counter += 1
if distract_counter >= DISTRACTION_FRAMES:
if distract_counter == DISTRACTION_FRAMES:
distraction_count += 1
cv2.putText(frame, "DISTRACTION ALERT",
(30, 90), cv2.FONT_HERSHEY_SIMPLEX,
1, (0, 165, 255), 3)
else:
distract_counter = 0
```

YAWN

```
if mar > MOUTH_THRESH:
yawn_counter += 1
if yawn_counter >= YAWN_FRAMES:
if yawn_counter == YAWN_FRAMES:
yawn_count += 1
cv2.putText(frame, "YAWNING DETECTED",
(30, 130), cv2.FONT_HERSHEY_SIMPLEX,
0.9, (255, 255, 0), 2)
else:
yawn_counter = 0
```

RISK LEVEL

```
total_events = drowsy_count + distraction_count
```

```
if total_events <= 1:
risk_text = "SAFE"
risk_color = (0, 255, 0)
elif total_events <= 3:
risk_text = "WARNING"
risk_color = (0, 255, 255)
else:
risk_text = "HIGH RISK"
risk_color = (0, 0, 255)

cv2.putText(frame, f"Risk Level: {risk_text}",
(300, 120), cv2.FONT_HERSHEY_SIMPLEX,
0.7, risk_color, 2)
```

DISPLAY

```
cv2.putText(frame, f"Drowsiness Count: {drowsy_count}",
(300, 30), cv2.FONT_HERSHEY_SIMPLEX,
0.6, (0, 255, 255), 2)

cv2.putText(frame, f"EAR: {ear:.2f}",
(300, 60), cv2.FONT_HERSHEY_SIMPLEX,
0.6, (255, 255, 255), 2)

cv2.putText(frame, f"MAR: {mar:.2f}",
(300, 90), cv2.FONT_HERSHEY_SIMPLEX,
0.6, (255, 255, 255), 2)

cv2.imshow("AI Drowsiness Detection System", frame)
```

```
if cv2.waitKey(1) & 0xFF == ord("q"):
session_end = time.time()
session_duration = session_end - session_start

minutes = int(session_duration // 60)
seconds = int(session_duration % 60)

timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

total_events = drowsy_count + distraction_count
if total_events <= 1:
overall_risk = "SAFE"
elif total_events <= 3:
overall_risk = "WARNING"
else:
overall_risk = "HIGH RISK"

summary = f"""
SESSION SUMMARY
Date & Time: {timestamp}
Drowsiness Events: {drowsy_count}
Distraction Events: {distraction_count}
Yawns Detected: {yawn_count}
Overall Risk Level: {overall_risk}
Session Duration: {minutes} min {seconds} sec

print(summary)
```

```
with open("session_report.txt", "a") as f:
f.write(summary)

break

cap.release()
cv2.destroyAllWindows()
```

DISTRACTION CODE

```
left_eye_center = np.mean(coords[LEFT_EYE], axis=0)
right_eye_center = np.mean(coords[RIGHT_EYE], axis=0)
eye_center_x = int((left_eye_center[0] + right_eye_center[0]) / 2)
nose_x = coords[NOSE_POINT][0]

if abs(nose_x - eye_center_x) > DISTRACTION_THRESH:
distract_counter += 1
if distract_counter >= DISTRACTION_FRAMES:
if distract_counter == DISTRACTION_FRAMES:
distraction_count += 1
cv2.putText(frame, "DISTRACTION ALERT",
(30, 90), cv2.FONT_HERSHEY_SIMPLEX,
1, (0, 165, 255), 3)
else:
distract_counter = 0
```

YAWN CODE

```
if mar > MOUTH_THRESH:
yawn_counter += 1
if yawn_counter >= YAWN_FRAMES:
```

```
if yawn_counter == YAWN_FRAMES:
yawn_count += 1
cv2.putText(frame, "YAWNING DETECTED",
(30, 130), cv2.FONT_HERSHEY_SIMPLEX,
0.9, (255, 255, 0), 2)
else:
yawn_counter = 0
```

WEBCAM CODE

```
cap = cv2.VideoCapture(0)
print("[INFO] Drowsiness + Distraction Detection Started")
while True:
ret, frame = cap.read()
if not ret:
break
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = detector(gray)
```

5.5 Code Validation and Optimization

The system was validated to ensure correct functionality of all modules.

Optimization techniques include:

- Using grayscale images for faster processing
- Efficient use of NumPy arrays
- Limiting computations to required facial regions
- Using consecutive frame validation to reduce false positives
- These optimizations improve system performance and accuracy.

5.6 Debugging

Debugging was performed to identify and fix errors during development.

Common issues resolved:

- Incorrect threshold values
- Indentation errors

- Library installation issues
- Alarm delay problems
- False detection of drowsiness and yawning
- Proper testing and debugging ensured stable system performance.

5.7 Unit Testing

Each module of the system was tested individually:

- Face detection module
- Eye detection module
- Yawn detection module
- Distraction detection module
- Alert system
- Logging system

This ensures each component functions correctly before integration.

5.8 Test Plan & Test Cases

The testing ensures that all features work correctly under real-time conditions and that the system responds appropriately to different inputs.

Test Plan

The system was tested under different conditions:

- Normal driving condition
- Eye closure (drowsiness simulation)
- Yawning simulation
- Head movement (distraction simulation)
- Face absence

Test Case

Test Case	Input Condition	Expected Output	Result
TC1	Eyes open	No alert	Pass
TC2	Eyes closed (1–2 sec)	Drowsiness alert	Pass
TC3	Mouth wide open	Yawning detected	Pass
TC4	Head turned	Distraction alert	Pass

TC5	Face not visible	Face not detected message	Pass
TC6	Press ‘Q’	Session summary generated	Pass
TC7	Multiple events	Risk level updated	Pass

SYSTEM TESTING

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

System testing is an essential phase in software development where the complete system is tested to ensure that it meets the specified requirements and functions correctly. It involves verifying the performance, accuracy, and reliability of the system under different conditions.

In the AI-Based Driver Drowsiness Detection System, testing is carried out to ensure that the system accurately detects drowsiness, yawning, and distraction in real time. The alert mechanisms, risk level indicator, and session logging features are also tested to confirm proper functionality. The system is evaluated under different user conditions to ensure stability and consistency.

6.2 Integration Testing

Integration testing is performed to verify that different modules of the system work together correctly. In this system, multiple modules such as face detection, landmark extraction, drowsiness detection, alert system, and logging system are integrated.

During testing, it is ensured that the output of one module correctly serves as the input to another. For example, the facial landmark detection module provides data to the EAR and MAR calculation modules, which then trigger alerts based on threshold values. The alert system works in coordination with the detection modules to generate sound and voice warnings.

The integration testing confirms that all components interact smoothly without errors or delays.

6.3 System Testing

System testing involves evaluating the complete system as a whole to ensure that it meets the required functionality. The system is tested in real-time conditions using a webcam.

Different scenarios are tested, such as normal driving conditions, eye closure, yawning, and head movement. The system successfully detects these behaviors and generates appropriate alerts. The “Face Not Detected” warning is also tested when the user is not visible in front of the camera.

The system demonstrates stable performance with accurate detection and real-time response, confirming that it meets the project objectives.

6.4 Test Plan & Test Cases

Test Plan

The system is tested under various conditions to evaluate its performance and accuracy. The test plan includes checking normal behavior, drowsiness detection, yawning detection, distraction detection, alert generation, and session logging.

The testing ensures that all features work correctly under real-time conditions and that the system responds appropriately to different inputs

Test Case

Test Case	Input Condition	Expected Output	Result
TC1	Eyes open	No alert	Pass
TC2	Eyes closed for few seconds	Drowsiness alert	Pass
TC3	Mouth wide open	Yawning detected	Pass
TC4	Head turned sideways	Distraction alert	Pass
TC5	Face not visible	Face not detected message	Pass
TC6	Multiple events	Risk level increases	Pass
TC7	Press ‘Q’	Session summary generated	Pass

SYSTEM MAINTAINANCE

CHAPTER 7

SYSTEM MAINTAINANCE

7.1 INTRODUCTION

System maintenance is an important phase in the software development lifecycle that ensures the system continues to function correctly after deployment. It involves updating, improving, and modifying the system to adapt to new requirements, fix errors, and enhance performance.

In the AI-Based Driver Drowsiness Detection System, maintenance ensures that the system remains accurate, efficient, and compatible with future technologies. Since the system is based on real-time processing and machine learning concepts, periodic updates are necessary to maintain its effectiveness.

In addition, continuous monitoring of the system is required to ensure that the detection algorithms perform accurately under different environmental conditions such as lighting variations, camera quality, and driver behavior. Maintenance also helps in improving user experience by making the system more responsive and reliable.

7.2 MAINTENANCE

Maintenance of the system can be categorized into different types:

Corrective Maintenance:

This involves fixing errors or bugs that may arise during system operation. For example, correcting false detection issues, improving threshold values, or resolving software errors.

Adaptive Maintenance:

This includes updating the system to work with new software environments, updated libraries, or different hardware configurations such as improved cameras.

Perfective Maintenance:

This focuses on enhancing system performance and adding new features. Examples include improving detection accuracy, optimizing processing speed, or adding new functionalities like advanced AI models or mobile integration.

Preventive Maintenance:

This involves making improvements to prevent future issues. It includes code optimization, updating dependencies, and improving system stability.

Regular maintenance ensures that the system remains reliable, accurate, and efficient in detecting driver drowsiness and preventing accidents.

SYSTEM SECURITY MEASURES

CHAPTER 8

SYSTEM SECURITY MEASURES

8.1 INTRODUCTION

System security is an important aspect of software development that ensures the system is protected from unauthorized access, data misuse, and potential threats. It involves implementing measures at different levels to safeguard system integrity, confidentiality, and reliability.

In the AI-Based Driver Drowsiness Detection System, security is maintained by ensuring safe execution of the application, protecting system resources, and preventing misuse of data. Since the system operates locally and does not store sensitive personal information, the risk of data breaches is minimal.

8.2 OPERATING SYSTEM LEVEL SECURITY

Operating system level security ensures that the application runs safely within the system environment.

The system is executed within a controlled environment such as Anaconda or a virtual environment, which isolates dependencies and prevents conflicts.

Access to system resources such as webcam and files is managed by the operating system, ensuring that only authorized applications can use them.

Antivirus and system security tools help prevent malicious software from affecting the application.

Proper user permissions ensure that unauthorized users cannot modify or misuse the system files.

These measures ensure that the application runs securely on the host system.

8.3 DATABASE LEVEL SECURITY

The system does not use a traditional database; instead, it stores session data in a local text file. Therefore, database-level security is minimal but still considered.

The session report file is stored locally, reducing exposure to external threats.

File access permissions can be set to restrict unauthorized modifications.

No sensitive personal data is stored, ensuring user privacy.

This approach simplifies security while maintaining data integrity.

8.4 SYSTEM LEVEL SECURITY

System-level security focuses on protecting the overall functioning of the application.

- The system does not require internet access, reducing vulnerability to cyber-attacks.

Real-time processing ensures that no unnecessary data is stored or transmitted.

- Input is limited to webcam video, preventing malicious data injection.
- Error handling and debugging practices improve system stability.
- The application avoids storing personal identity data, ensuring privacy protection.

These measures ensure that the system is secure, reliable, and safe for practical use.

SYSTEM PLANNING AND SCHEDULING

CHAPTER 9

SYSTEM PLANNING AND SCHEDULING

9.1 INTRODUCTION

System planning and scheduling are essential activities in software development that ensure the project is completed efficiently within the given time and resources. Planning involves identifying tasks, allocating resources, and defining timelines, while scheduling organizes these tasks in a logical sequence.

In the AI-Based Driver Drowsiness Detection System, proper planning helped in organizing development stages such as requirement analysis, design, implementation, testing, and documentation. This ensured smooth execution and timely completion of the project.This project focuses on how ai can be used to detect drowsiness.

9.2 PLANNING A SOFTWARE PROJECT

Planning a software project involves defining objectives, identifying requirements, and organizing tasks in a systematic manner.

For this project, planning included:

- Understanding the problem of driver drowsiness and accident prevention
- Identifying system requirements and features
- Selecting appropriate technologies such as Python, OpenCV, and dlib
- Designing system modules like detection, alert system, and logging
- Allocating time for coding, testing, and debugging
- Preparing documentation and final presentation
- Effective planning ensured that the project progressed in an organized and structured way.

9.3 STEPS INVOLVED IN PLANNING A SYSTEM

The following steps were involved in planning the system:

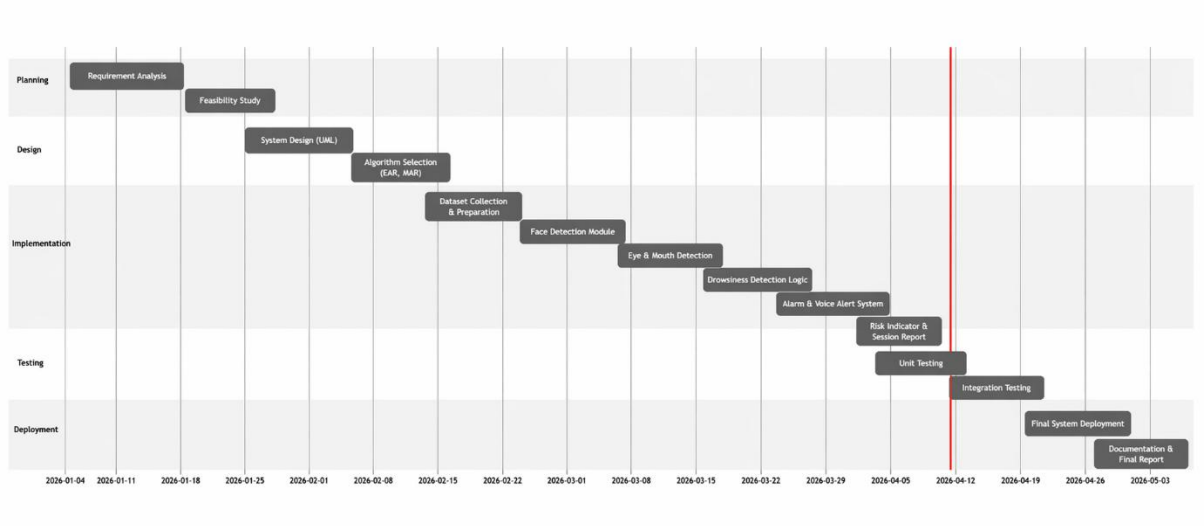
- Requirement Analysis:
- Identifying the need for a real-time drowsiness detection system and defining its features.
- System Design:
- Designing modules such as face detection, drowsiness detection, alert system, and logging.
- Technology Selection:
- Choosing Python and relevant libraries for implementation.
- Implementation:

- Developing the system using coding and integrating different modules.
- Testing:
- Verifying system functionality under different conditions.
- Documentation:
- Preparing detailed project documentation.
- Deployment:
- Running the system in real-time environment for demonstration.

9.4 GANTT CHART

A Gantt chart is a visual representation of the project schedule that shows task durations and their sequence over time.

The following Gantt chart represents the development timeline of the AI Drowsiness System:

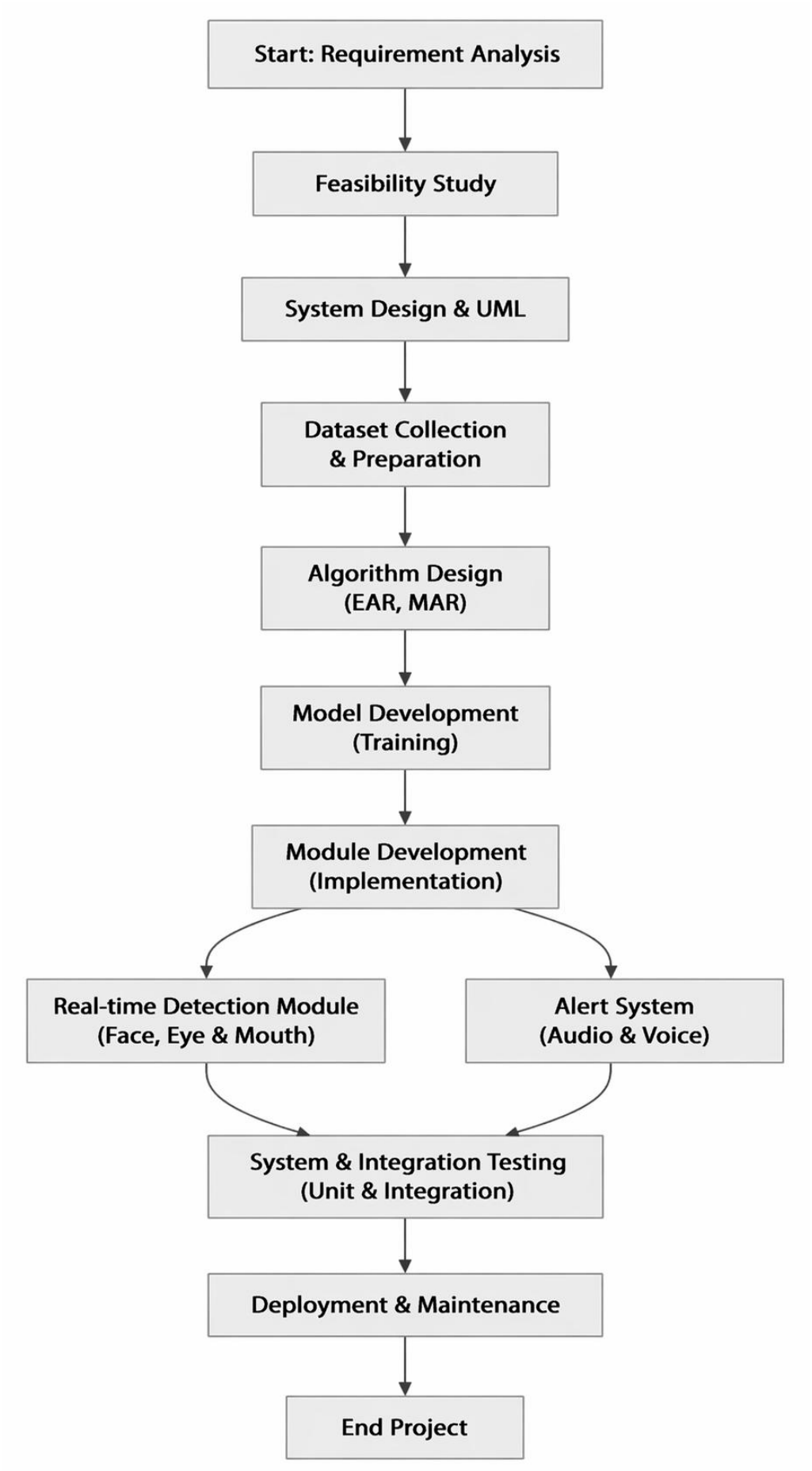


9.5 PERT CHART

A PERT (Program Evaluation and Review Technique) chart is used to analyze task dependencies and determine the critical path in a project. T

he PERT chart highlights that backend API development acts as a critical stage, as both frontend integration and cloud storage implementation depend on it before system testing can begin.

The following PERT chart illustrates the sequence of activities in the AI Drowsiness System


Start: Requirement Analysis
Feasibility Study
System Design & UML
Dataset Collection & Preparation
Algorithm Design (EAR, MAR)
Model Development (Training)
Module Development (Implementation)
Real-time Detection Module (Face, Eye & Mouth)
Alert System (Audio & Voice)
System & Integration Testing (Unit & Integration)
Deployment & Maintenance
End Project

SYSTEM COST ESTIMATION

CHAPTER 10

SYSTEM COST ESTIMATION

10.1 INTRODUCTION

System cost estimation is the process of predicting the effort, time, and cost required to develop a software system. It helps in understanding the resources needed and planning the development process effectively.

In the AI-Based Driver Drowsiness Detection System, cost estimation is performed using the Lines of Code (LOC) based estimation method. Since the project is developed using Python and does not require expensive hardware or external services, the overall cost is minimal.

10.2 LOC Based Estimation

Lines of Code (LOC) based estimation is a technique where the size of the software is measured based on the number of lines of code written. The effort and cost are then estimated based on this size.

For this project:

- Estimated number of lines of code ≈ 400–600 lines
- Programming language used: Python

Effort Estimation

The effort required to develop the system includes:

- Requirement analysis
- System design
- Coding and implementation
- Testing and debugging
- Documentation

The total development effort is approximately 4–6 weeks.

Cost Estimation

Since the project uses open-source tools and libraries, the development cost is low.

Component	**Cost**
Software (Python, OpenCV, dlib)	Free
Development Tools (VS Code, Anaconda)	Free
Hardware (Laptop, Webcam)	Already available
Development Effort	Student work
Total Cost	Minimal / Negligible

CONCLUSION OF ESTIMATION

The LOC-based estimation shows that the system is cost-effective and feasible. The use of open-source technologies reduces financial requirements while still delivering a functional and efficient system.

FUTURE ENHANCEMENT AND SCOPE OF FURTHER DEVELOPMENT

CHAPTER 11

FUTURE ENHANCEMENT AND SCOPE OF FURTHER DEVELOPMENT

11.1 INTRODUCTION

This chapter discusses the advantages, limitations, and future improvements of the AI-Based Driver Drowsiness Detection System. While the current system effectively detects driver fatigue using computer vision techniques, there is always scope for enhancement to improve accuracy, performance, and real-world applicability.

11.2 MERITS OF THE SYSTEM

The proposed system offers several advantages:

- The system operates in real time, providing immediate detection and alerts.
- It is a cost-effective solution as it does not require additional hardware.
- The system is non-intrusive and does not require physical contact with the user.
- It uses Artificial Intelligence and Computer Vision for intelligent analysis.
- It includes multiple detection features such as drowsiness, yawning, and distraction.
- It provides both alarm and voice alerts for better effectiveness.
- The risk level indicator helps in analyzing overall driver condition.
- The session logging feature allows tracking of driver behavior over time.
- The system is easy to use and can run on standard devices.

11.3 LIMITATIONS OF THE SYSTEM

Despite its advantages, the system has some limitations:

- The system depends on proper lighting conditions for accurate detection.
- It requires the driver's face to be clearly visible at all times.
- Performance may be affected by low-quality cameras.
- The system uses a frontal face detector, which may fail when the head is turned significantly.
- It does not include automatic vehicle control mechanisms.
- The system may produce minor false detections under certain conditions.

11.4 FUTURE ENHANCEMENT OF THE SYSTEM

The system can be further improved in the following ways:

Integration with deep learning models for improved accuracy.Implementation of head pose estimation for better distraction detection.

- Integration with IoT-enabled smart vehicles.
- Automatic vehicle control features such as speed reduction or braking.
- Development of a mobile application version of the system.
- Cloud-based data storage and analysis for long-term monitoring.
- Use of infrared cameras for better performance in low-light conditions.
- Addition of GPS-based emergency alert system.

CONCLUSION

CHAPTER 12

CONCLUSION

12.1 CONCLUSION

The AI-Based Driver Drowsiness Detection System successfully demonstrates the use of Artificial Intelligence and Computer Vision techniques to improve road safety. The system is capable of monitoring driver behavior in real time using a webcam and detecting signs of drowsiness, yawning, and distraction effectively.

By calculating Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), the system identifies fatigue-related patterns and provides timely alerts through alarm sound and voice notifications. The inclusion of a risk level indicator further enhances the system by analyzing cumulative driver behavior during a session.

Additionally, the system generates a session summary report, which provides useful insights into driver activity and performance. The implementation is cost-effective, as it does not require additional hardware, and can be easily deployed on standard computing devices.

Although the system has certain limitations such as dependency on lighting conditions and face visibility, it serves as a reliable and practical solution for real-time driver monitoring. With further improvements and integration into smart vehicle systems, this project has the potential to contribute significantly to accident prevention and road safety.

BIBLIOGRAPHY

CHAPTER 13

BIBLIOGRAPHY

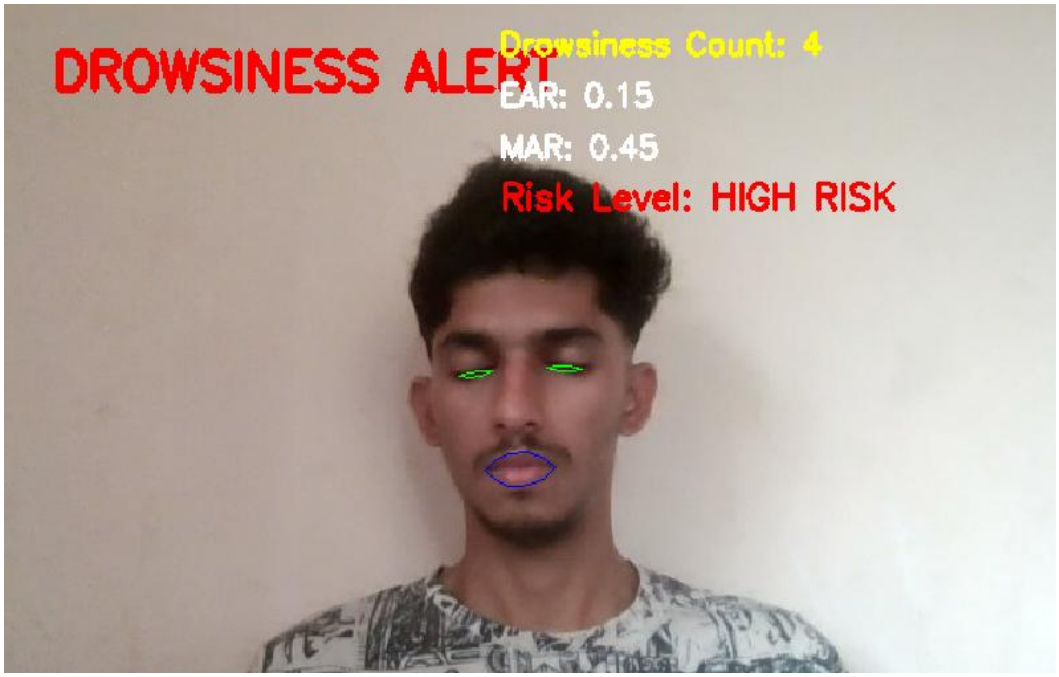
13.1 REFERENCES

- OpenCV, “Open Source Computer Vision Library,” [Online]. Available: https://opencv.org/
- D. E. King, “Dlib-ml: A Machine Learning Toolkit,” Journal of Machine Learning Research, vol. 10, pp. 1755–1758, 2009.
- Python Software Foundation, “Python Documentation,” [Online]. Available: https://docs.python.org/3/
- NumPy Developers, “NumPy Documentation,” [Online]. Available: https://numpy.org/
- SciPy Community, “SciPy Documentation,” [Online]. Available: https://scipy.org/
- pyttsx3 Documentation, “Text-to-Speech Library in Python,” [Online]. Available: https://pyttsx3.readthedocs.io/
- Mosh Hamedani, “Python Full Course for Beginners,” YouTube, Programming with Mosh.
- T. Soukupová and J. Čech, “Real-Time Eye Blink Detection using Facial Landmarks,” Computer Vision Winter Workshop, 2016.
- GitHub, “Driver Drowsiness Detection Projects,” [Online]. Available: https://github.com

APPENDIX

CHAPTER 14

APPENDIX




Drowsiness Count: 1
EAR: 0.27
MAR: 0.32
Risk Level: WARNING